

МЛИТА

МАТЕМАТИЧЕСКАЯ ЛОГИКА И ТЕОРИЯ АЛГОРИТМОВ

Математическая логика

1. Введение в математическую логику
2. Основы теории множеств
3. Булевы функции
4. Дизъюнктивная и конъюнктивная нормальные формы
5. Совершенные дизъюнктивная и конъюнктивная нормальные формы
6. Классы функций
7. Полнота система булевых функций
8. Предикаты

Теория алгоритмов

9. Введение в теорию алгоритмов
10. Основные понятия алгоритмов
11. Основы реализации алгоритмов
12. Линейные алгоритмы
13. Нелинейные алгоритмы
14. Основы сложности алгоритмов
15. Оценка сложности алгоритмов (продолжение)
16. Виды алгоритмов

1. Введение в математическую логику

Математическая логика: что это и зачем?

Определение и цель

Логика — это наука о правильном мышлении, рассуждениях и доказательствах. Математическая логика формализует эти процессы.

Пропозициональная Логика

Работает с простыми утверждениями (пропозициями), которые могут быть либо истинными (1), либо ложными (0).

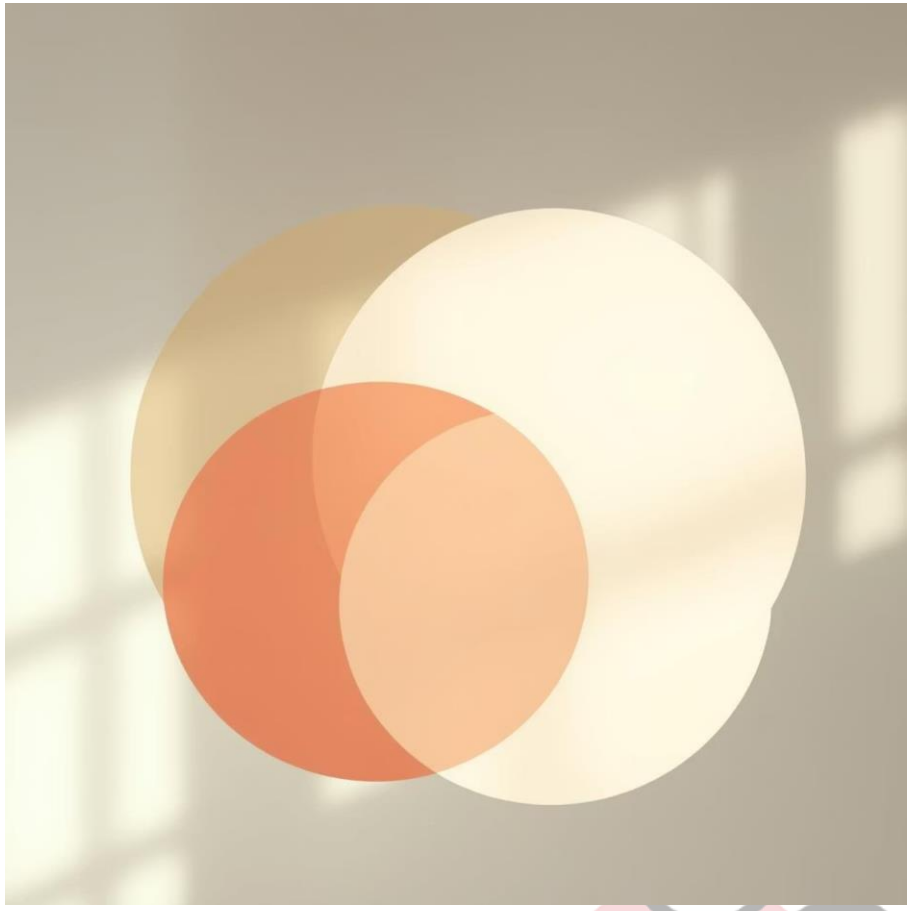
Основные Операции

Фундаментальные логические операции включают отрицание (**НЕ**), конъюнкцию (**И**), дизъюнкцию (**ИЛИ**), импликацию (**ЕСЛИ...ТО**) и эквивалентность (**ТОГДА И ТОЛЬКО ТОГДА**).

Математическая логика обеспечивает строгий язык для описания и анализа рассуждений, что критически важно в компьютерных науках, искусственном интеллекте и философии.

2. Основы теории множеств

- **Множество**: Коллекция уникальных, не упорядоченных объектов, обычно обозначаемая фигурными скобками {}. Например, A = {1, 2, 3}.
- **Определение множеств**: Элементы могут быть перечислены или заданы характеристическим свойством, например, {x | x - чётное число}.
- **Специальные множества**: **пустое множество** ∅ (не содержит элементов) и **универсальное множество** U (все возможные элементы в данном контексте).



Теория множеств — это фундаментальный язык всей математики, позволяющий строго определить понятия чисел, функций и отношений.

Операции над множествами:

- **Объединение** (A ∪ B): Все элементы, принадлежащие A или B.
- **Пересечение** (A ∩ B): Все элементы, принадлежащие A и B.
- **Дополнение** (A'): Все элементы в универсальном множестве, не принадлежащие A.

3. Булевы функции: логика в алгебраической форме

Определение

Булева функция — это отображение, которое принимает n двоичных входов (0 или 1) и производит один двоичный выход (0 или 1).

Таблицы Истинности

Способ полного описания поведения булевой функции путем перечисления всех возможных комбинаций входов и соответствующих им выходов.

Количество Функций

Для n переменных существует 2^{2^n} различных булевых функций. Например, для 2 переменных их 16.

Булевы функции — это математическая модель логических вентилей в цифровых схемах и основы программирования.

4. Дизъюнктивная и конъюнктивная нормальные формы (ДНФ и КНФ)

ДНФ (Дизъюнктивная Нормальная Форма)

Выражение, представляющее собой **дизъюнкцию** (логическое ИЛИ) одного или нескольких **конъюнктов**, где каждый конъюнкт — это **конъюнкция** (логическое И) одной или нескольких переменных или их отрицаний (литералов).

- Пример: $(x \land \neg y) \lor (\neg x \land z)$

КНФ (Конъюнктивная Нормальная Форма)

Выражение, представляющее собой **конъюнкцию** (логическое И) одного или нескольких **дизъюнктов**, где каждый дизъюнкт — это **дизъюнкция** (логическое ИЛИ) одной или нескольких переменных или их отрицаний (литералов).

- Пример: $(x \lor y) \land (\neg x \lor z)$

ДНФ и КНФ являются стандартными формами для логических выражений, используемыми для анализа, упрощения и построения логических схем.



5. Совершенные дизъюнктивная и конъюнктивная нормальные формы

Совершенная ДНФ (СДНФ)

Это дизъюнкция всех **минтермов** (элементарных конъюнкций), для которых функция принимает значение 1. Каждый минтерм содержит все переменные функции.

- Построение СДНФ: **1.** Составить таблицу истинности. **2.** Выбрать строки, где функция равна 1. **3.** Для каждой такой строки записать минтерм. **4.** Объединить все минтермы дизъюнкцией.

Совершенная КНФ (СКНФ)

Это конъюнкция всех **макстермов** (элементарных дизъюнкций), для которых функция принимает значение 0. Каждый макстерм также содержит все переменные функции.

- Построение СКНФ: **1.** Составить таблицу истинности. **2.** Выбрать строки, где функция равна 0. **3.** Для каждой такой строки записать макстерм. **4.** Объединить все макстермы конъюнкцией.

СДНФ и СКНФ уникальны для каждой булевой функции и используются для минимизации логических выражений и синтеза логических схем.



6. Классы булевых функций



Монотонные функции

Если изменение входных переменных с 0 на 1 не приводит к изменению выходного значения с 1 на 0. Это функции, которые можно выразить без отрицаний.



Самодвойственные функции

Функция $f(x_1, \dots, x_n)$ является самодвойственной, если $\neg f(x_1, \dots, x_n) = f(\neg x_1, \dots, \neg x_n)$.



Линейные функции

Функции, которые можно представить в виде суммы по модулю 2 (XOR) входных переменных и к ст ты. Например, $f(x, y) = x \oplus y \oplus 1$.

Классификация булевых функций помогает упрощать анализ и синтез сложных логических схем и алгоритмов.



7. Полнота систем булевых функций

Что такое полнота?

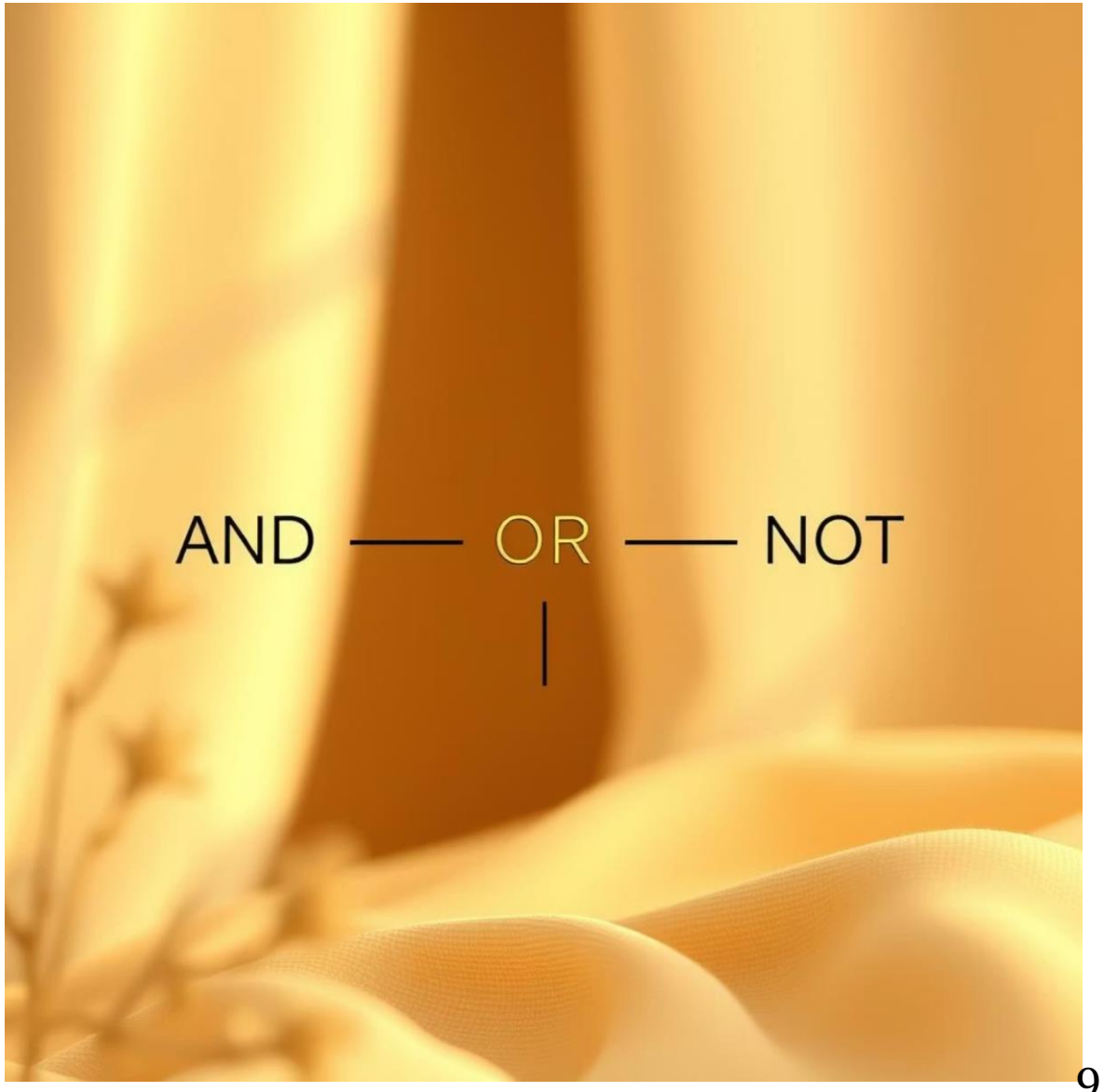
Система булевых функций называется полной, если с её помощью можно выразить любую другую булеву функцию.

Это означает, что имея лишь элементы этой системы (например, логические вентили), можно построить любую желаемую логическую схему.

Это фундаментальное понятие в схемотехнике и теории вычислимости.

Примеры полных систем:

- {AND, OR, NOT} - классический набор, который позволяет реализовать любую логическую функцию.
- {NOR} - аналогично, функция **ИЛИ-НЕ** также является универсальной.
- {NAND} - одна лишь функция **И-НЕ** является универсальной (полной). Используя только вентили **И-НЕ**, можно построить **И**, **ИЛИ**, **НЕ** и, следовательно, любую другую функцию.



8. Предикаты и предикатная логика

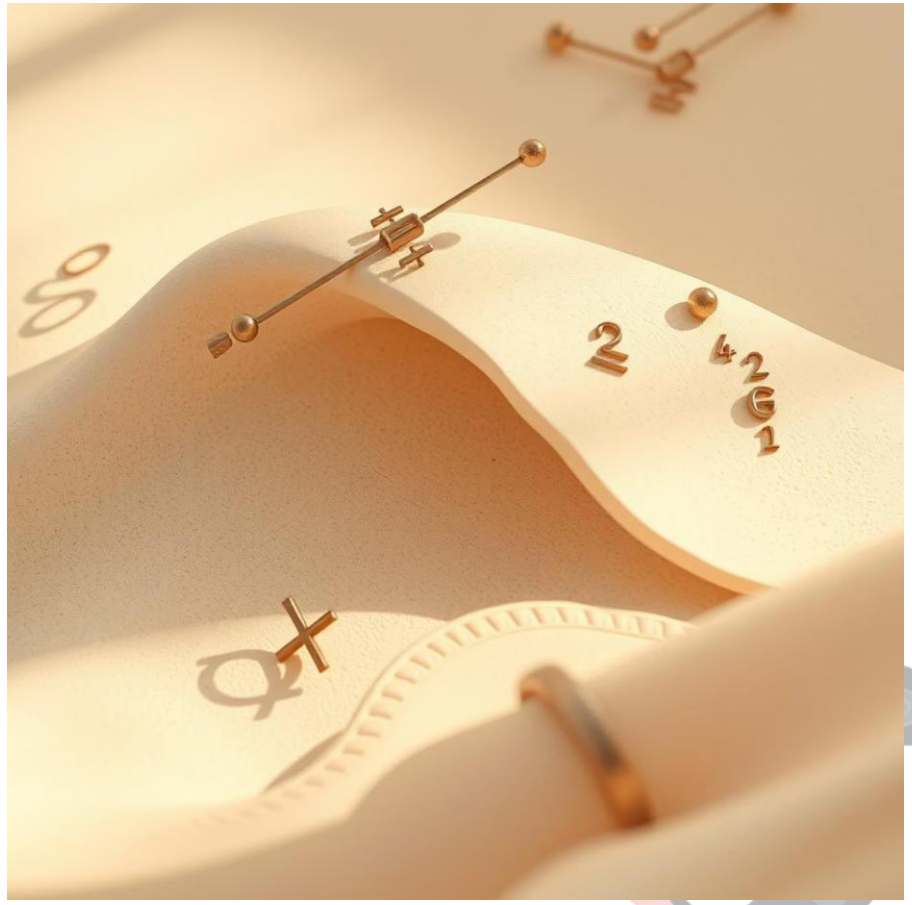
"Логика предикатов позволяет выражать отношения и свойства, которые невозможно представить в пропозициональной логике."

Предикат — это высказывательная форма, содержащая одну или несколько переменных, которая становится истинной или ложной при подстановке конкретных значений вместо этих переменных. Например, "$x > 5$" или "x является студентом y".

Кванторы:

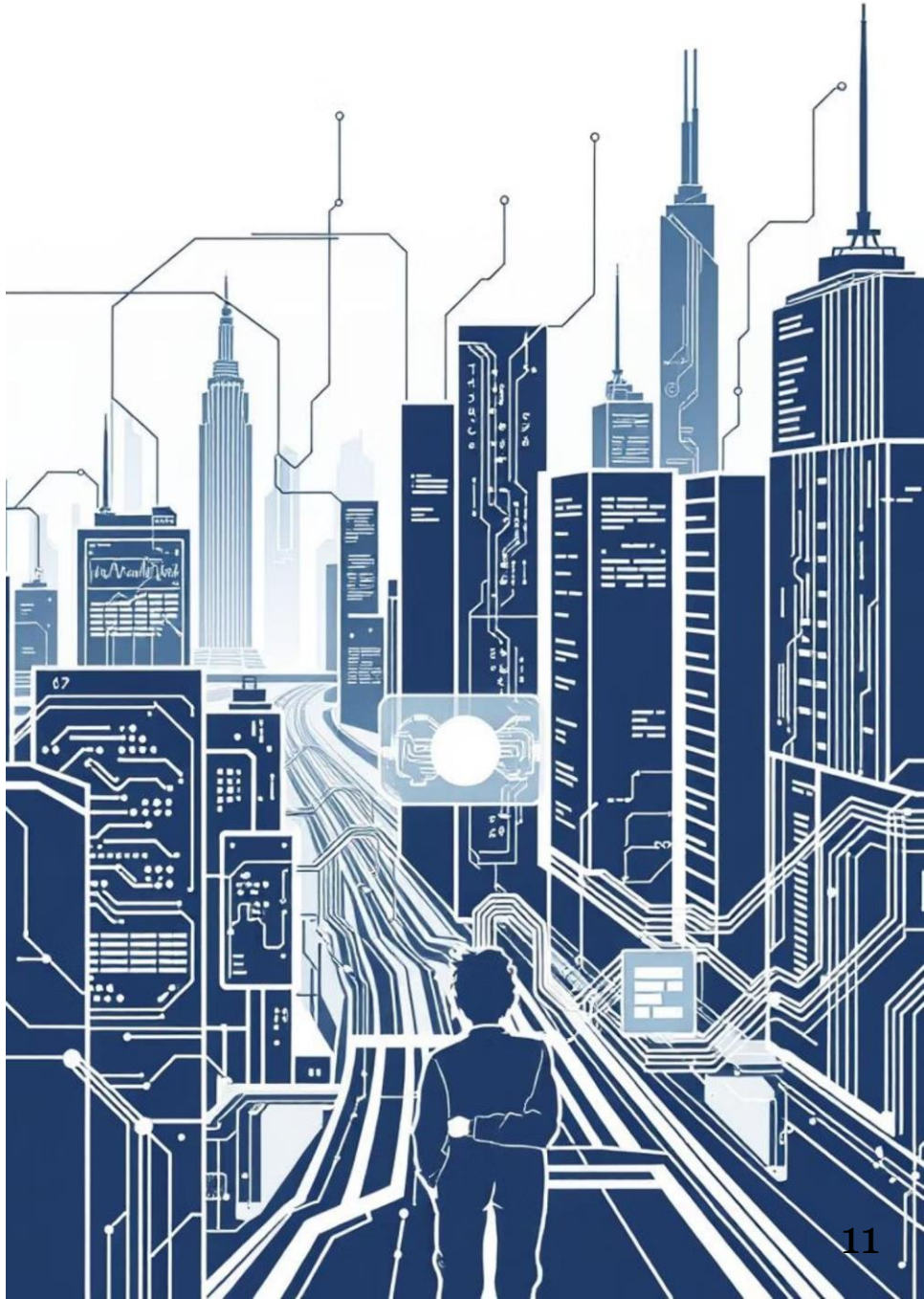
- **Квантор всеобщности** (∀, "для любого", "для всех"): Утверждает, что свойство выполняется для всех элементов в определённой области.
- Например, $\forall x\ (x > 0)$ — "Для любого x, x больше 0".
- **Квантор существования** (∃, "существует", "найдётся"): Утверждает, что свойство выполняется хотя бы для одного элемента в определённой области. Например, $\exists y(y < 10)$ — "Существует y, такое что y меньше 10".

Предикатная логика (логика первого порядка) — это мощное расширение пропозициональной логики, позволяющее моделировать сложные отношения и делать выводы о свойствах объектов.



9. Введение в теорию алгоритмов

От основ к пониманию эффективности



10. Основные понятия алгоритмов

Что такое алгоритм?

Алгоритм — это чёткая, конечная последовательность действий, выполнение которой приводит к решению поставленной задачи. Это как рецепт, но для компьютера.

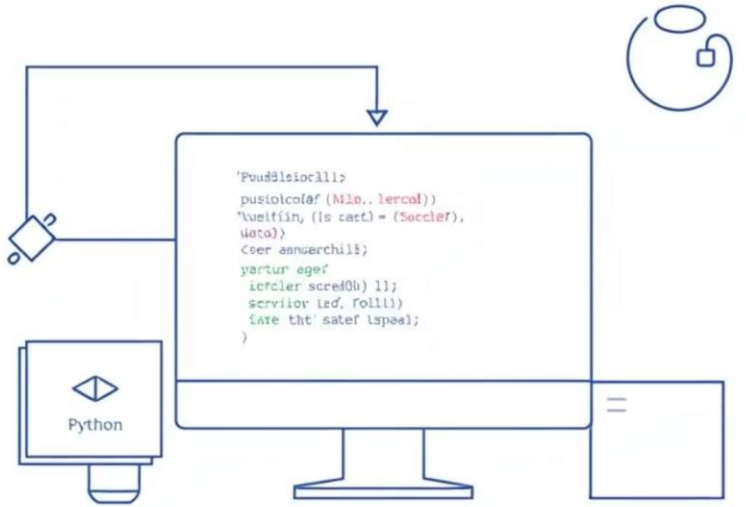
Ключевые свойства

- **Дискретность:** состоит из отдельных, чётких шагов.
- **Определённость:** каждый шаг однозначно определён.
- **Результативность:** завершается за конечное число шагов с определённым результатом.
- **Массовость:** применим для целого класса задач.

Пример на практике

Рассмотрим алгоритм сортировки массива чисел — это набор инструкций, который упорядочит любые числа, будь их 10 или 10 миллионов.

11. Основы реализации алгоритмов



Как представить алгоритм?

Алгоритмы можно описывать по-разному:

- **Псевдокод:** человекочитаемый, но структурированный язык.
- **Блок-схемы:** графическое представление шагов.
- **Программный код:** реализация на конкретном языке программирования, например, на Python.

Роль структур данных

Выбор правильной структуры данных (например, массив, список, дерево) критически важен для эффективности алгоритма. Неверный выбор может сделать даже самый элегантный алгоритм медленным.

Пример: def linear_search(arr, target):for i in range(len(arr)):if arr[i] == target:return ireturn -1 Это простая реализация линейного поиска на Python.

12. Линейные алгоритмы

Линейные алгоритмы характеризуются простым, последовательным выполнением шагов. В них нет ветвлений (if/else) или циклов (for/while), которые бы меняли поток выполнения.

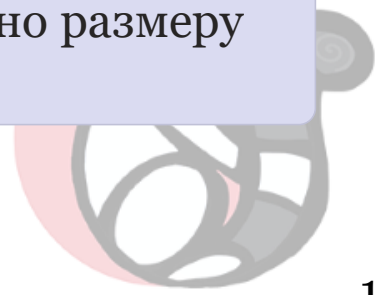
Пример: алгоритм подсчёта суммы всех элементов массива. Чтобы посчитать сумму 10 элементов, нужно 10 раз сложить. Чтобы посчитать 100 элементов, нужно 100 раз сложить.

Прямой путь

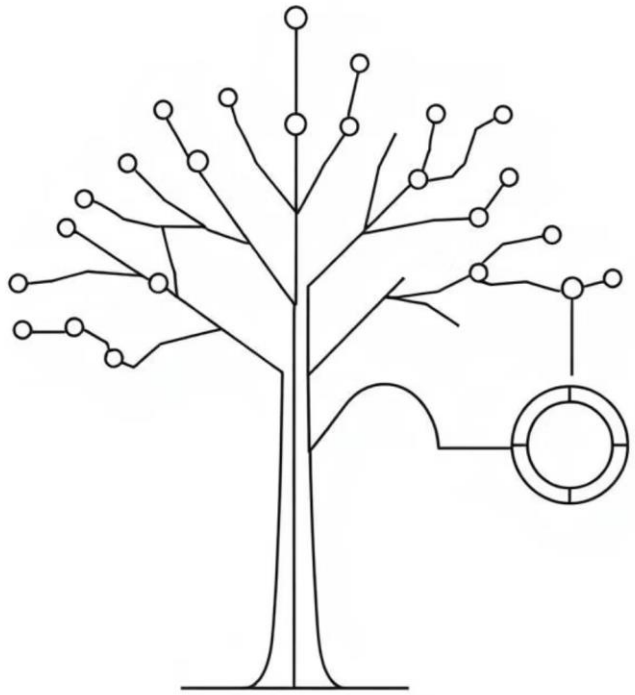
Каждая операция выполняется ровно один раз, друг за другом, без пропусков или повторений.

Временная сложность

Как правило, временная сложность таких алгоритмов составляет $O(1)$ (константная) или $O(n)$ (линейная), если количество операций прямо пропорционально размеру входных данных.



13. Нелинейные алгоритмы



Нелинейные алгоритмы — это те, которые включают в себя более сложные структуры управления потоком:

- **Ветвления:** использование условных операторов (`if`, `else if`, `else`) для выполнения разных действий в зависимости от условий.

- **Циклы:** многократное повторение блока кода (`for`, `while`) до выполнения определённого условия.
- **Рекурсия:** функция, вызывающая саму себя для решения подзадач.

Сложность таких алгоритмов может варьироваться от $O(\log n)$ (очень быстрая) до экспоненциальной ($O(2^n)$), что делает их менее масштабируемыми при больших объёмах данных.

Примеры: бинарный поиск (быстрый, использует ветвления), рекурсивные алгоритмы (например, вычисление чисел Фибоначчи).

14. Основы сложности алгоритмов

Время выполнения

Сколько времени потребуется алгоритму для завершения работы в зависимости от размера входных данных?

Потребление памяти

Сколько оперативной памяти потребуется алгоритму для хранения данных и промежуточных результатов?

Big *O* нотация

Используется для описания того, как время выполнения или потребление памяти алгоритма растёт с увеличением размера входных данных (n). Она позволяет оценить производительность в худшем случае.

Понимание Big *O* помогает предсказывать, как алгоритм поведёт себя при обработке очень больших объёмов данных и выбирать наиболее эффективное решение.



15. Оценка сложности алгоритмов (продолжение)

O(1)

Константная сложность. Время не зависит от размера данных.

O(n)

Линейная сложность. Время растёт пропорционально размеру данных.

$O(n^2)$

Квадратичная сложность. Время растёт квадратично, становится очень медленным при больших n.

$O(2^n)$

Экспоненциальная сложность. Время растёт катастрофически быстро, применимо только для очень малых n.

Временная vs. Пространственная

- **Временная сложность:** сколько операций (и времени) нужно.
- **Пространственная сложность:** сколько памяти нужно.

Удвоение входных данных может означать совсем разный рост времени выполнения для алгоритмов с разной сложностью! **Например:** для $O(n)$ время удвоится, а для $O(n^2)$ — увеличится в 4 раза.



16. Виды алгоритмов

По способу решения

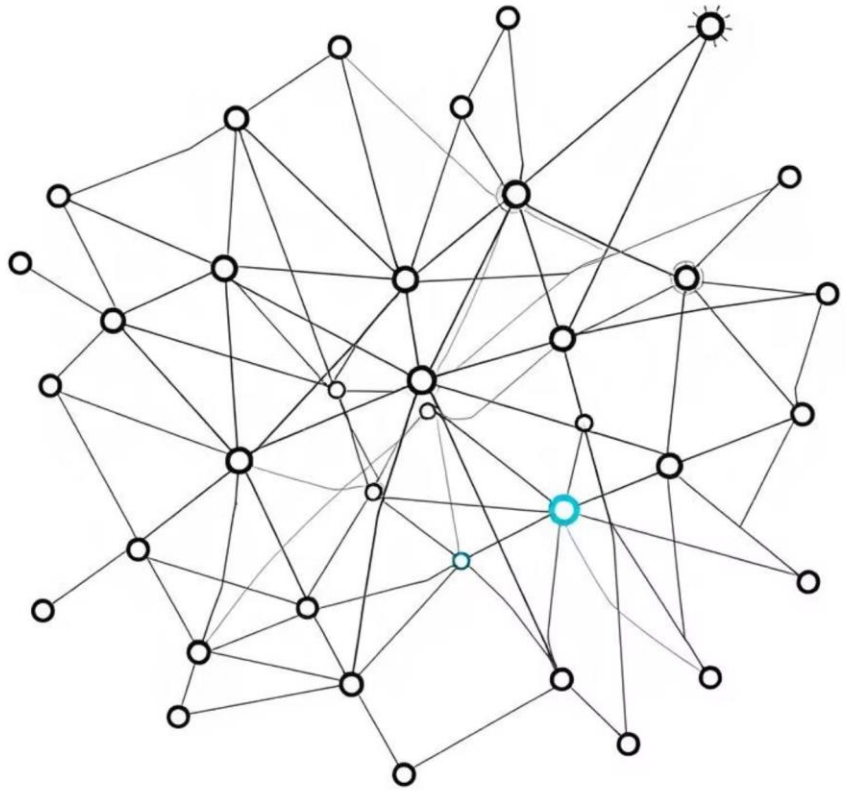
- **Жадные алгоритмы:** на каждом шаге выбирают локально лучшее решение, надеясь получить глобально оптимальное. Пример: Алгоритм Дейкстры (поиск кратчайшего пути).

- **Динамическое программирование:** разбивают задачу на перекрывающиеся подзадачи и сохраняют результаты, чтобы не пересчитывать их. Пример: Задача о рюкзаке.

- **Разделяй и властвуй:** разбивают задачу на независимые подзадачи, решают их и объединяют результаты. Пример: Сортировка слиянием.

По структуре

- **Итеративные:** используют циклы (for, while) для повторения операций. Пример: Итеративный факториал.
- **Рекурсивные:** функция вызывает саму себя для решения подзадач. Пример: Вычисление чисел Фибоначчи.



Логика и интуиция: два столпа мышления

Мыслительная деятельность человека — это сложный и многогранный процесс, происходящий как на сознательном, так и на бессознательном уровнях. Это высшая ступень человеческого познания, способствующая нахождению истины. Давайте погрузимся в мир этих фундаментальных аспектов нашего интеллекта.



Логика vs. интуиция

Логическое мышление

Логика (дедуктивное мышление) — это процесс, который от истинных посылок всегда приводит к истинному заключению. Оно не опирается на опыт, интуицию или другие внешние факторы, протекая полностью на уровне сознания.

Интуитивное познание

Интуиция (от лат. intuitio – «пристальное всматривание») — это способность постижения истины путем прямого ее усмотрения без логически строгого доказательства. Она является антиподом логики, действуя на подсознательном уровне.

Термин «логика» происходит от греч. «логос» — «слово», «понятие», «смысл».

Интуиция — это своего рода «озарение» или «прозрение», результат внелогического процесса.



Неразрывная связь

Сознание и логика

Логические процессы протекают на сознательном уровне.



Подсознание и интуиция

Интуитивное мышление происходит на подсознательном уровне.

Взаимодействие

Эти два аспекта мышления неразрывно связаны и необходимы для глубокого познания.

Вопрос о противопоставлении логического и интуитивного давно превратился в проблему взаимодействия этих двух ипостасей человеческого сознания в процессе познания.

Роль интуиции в науке

Развитие науки, особенно математики, немыслимо без интуиции. Различают два вида интуиции в научном познании: интуицию-суждение и интуицию-догадку.

Интуиция-суждение

Прямое усмотрение истины без логически строгого доказательства, которого для данной истины не существует и не может существовать в принципе. Это единовременный, целостный акт обобщающего характера.

Интуиция-догадка

Прямое внелогическое усмотрение факта, который впоследствии будет обоснован и доказан строго логическим путем. Проявляется как «озарение».



Интуиция-суждение: недоказуемая истина

Этот вид интуиции используется для установления истинности положений, помещаемых в основание той или иной математической теории, которая затем будет развиваться строгими формально-логическими методами.

> Примером являются логически недоказуемые утверждения, такие как тезисы Тьюринга, Чёрча и Маркова в теории алгоритмов.

Именно они закладывают фундаментальные основы, которые затем подтверждаются косвенными методами.



Интуиция-догадка: предвестник доказательства



Интуиция-догадка, или психоэвристическая интуиция, характеризуется внелогическим усмотрением факта, который впоследствии будет строго логически доказан. Это происходит бессознательно или подсознательно в короткие промежутки времени.

- Проявляется как «озарение» или «прозрение».
- Последующее логическое доказательство может занимать часы, дни и даже годы.

Такие озарения часто становятся отправной точкой для больших открытий.

Синергия логики и интуиции

Гипертрофия роли как строгой логики, так и интуиции — это крайности. Для познания мира необходимы оба метода:

Строгая Логика

Доказательный метод, обеспечивающий достоверность.

Интуитивный Подход

Основан на непосредственном синтетическом суждении, не опирающемся на доказательство.

Гармония

«Вся сила математики – в разумном сочетании интуиции и строгости. Контролируемый дух и вдохновенная логика!» - Я. Стюарт

Традиционная (формальная) логика

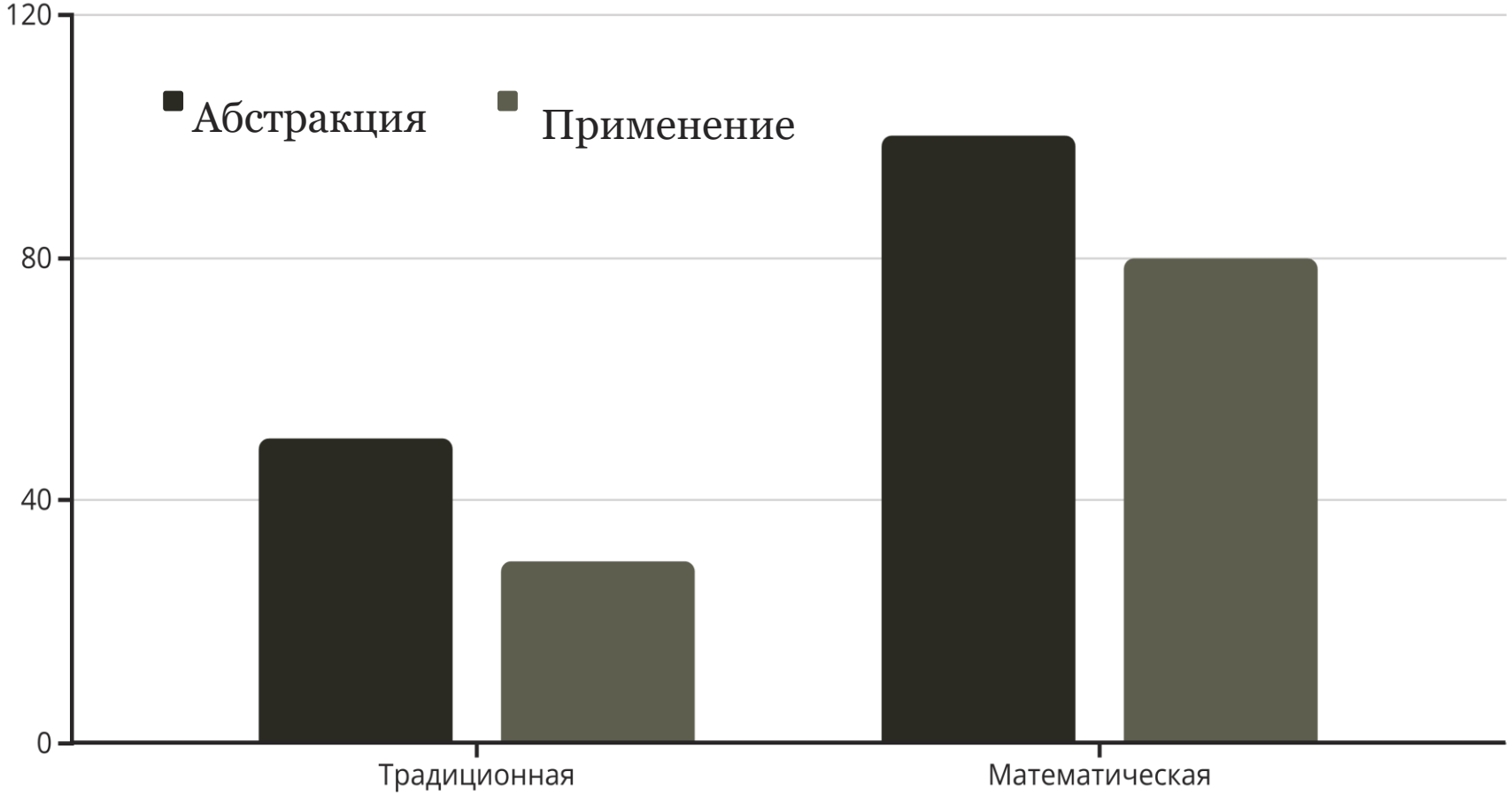
Традиционная логика, берущая свое начало от Аристотеля, изучает формы и законы мышления, а также методы, с помощью которых люди делают выводы.

> А. Пуанкаре: «Таким образом, логика и интуиция играют каждая свою необходимую роль. Обе они неизбежны. Логика, которая одна может дать достоверность, есть орудие доказательства; интуиция есть орудие изобретательства».

Формирование законов мышления — результат длительной общественной практики и познания окружающей действительности. Логика — это способ отражения действительности и наука о законах и формах правильного мышления.



Математическая (символическая) логика



Математическая логика выросла из традиционной, значительно расширив ее. Она применяет математические методы для изучения структур мышления и процесса доказательства математических теорем.

- **Более общая и абстрактная**: Изучает общие структуры правильного мышления.
- **Более конкретная**: Позволяет решать множество практических задач, недоступных для традиционной логики.
- **Инструмент для исследований**: В области оснований математики (теория доказательств, метаматематика).

Ключевые выводы и перспективы

Взаимодополняемость 1

Логика и интуиция — не антагонисты, а взаимодополняющие силы, необходимые для полноценного мышления и познания.

Роль в Науке 2

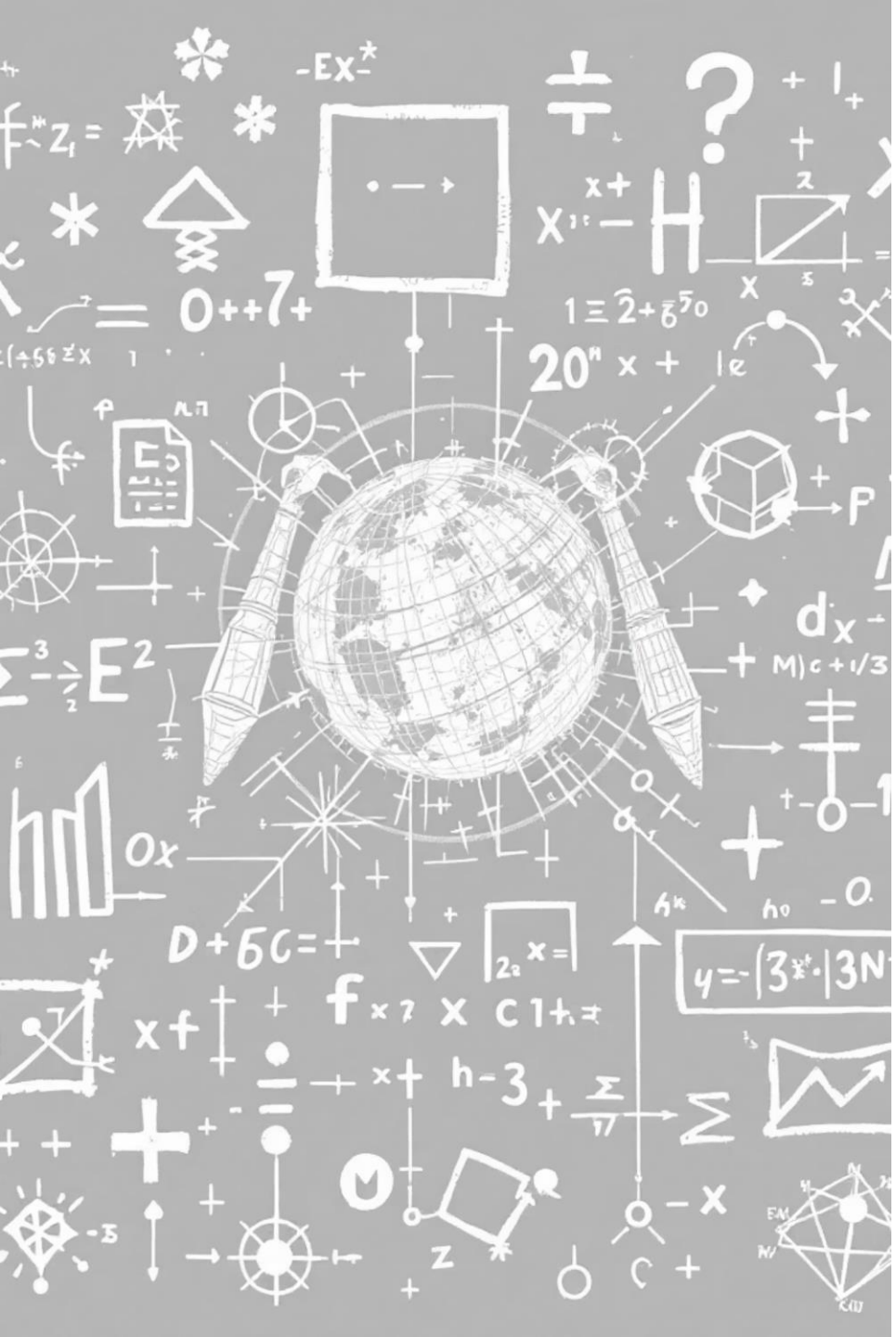
Интуиция критически важна для научных открытий, часто предваряя строго логические доказательства.

Эволюция Логики 3

Математическая логика расширяет традиционную, предоставляя более мощные инструменты для решения сложных задач.

Понимание этой синергии позволяет развивать мышление и эффективнее применять его как в теоретических, так и в практических областях.





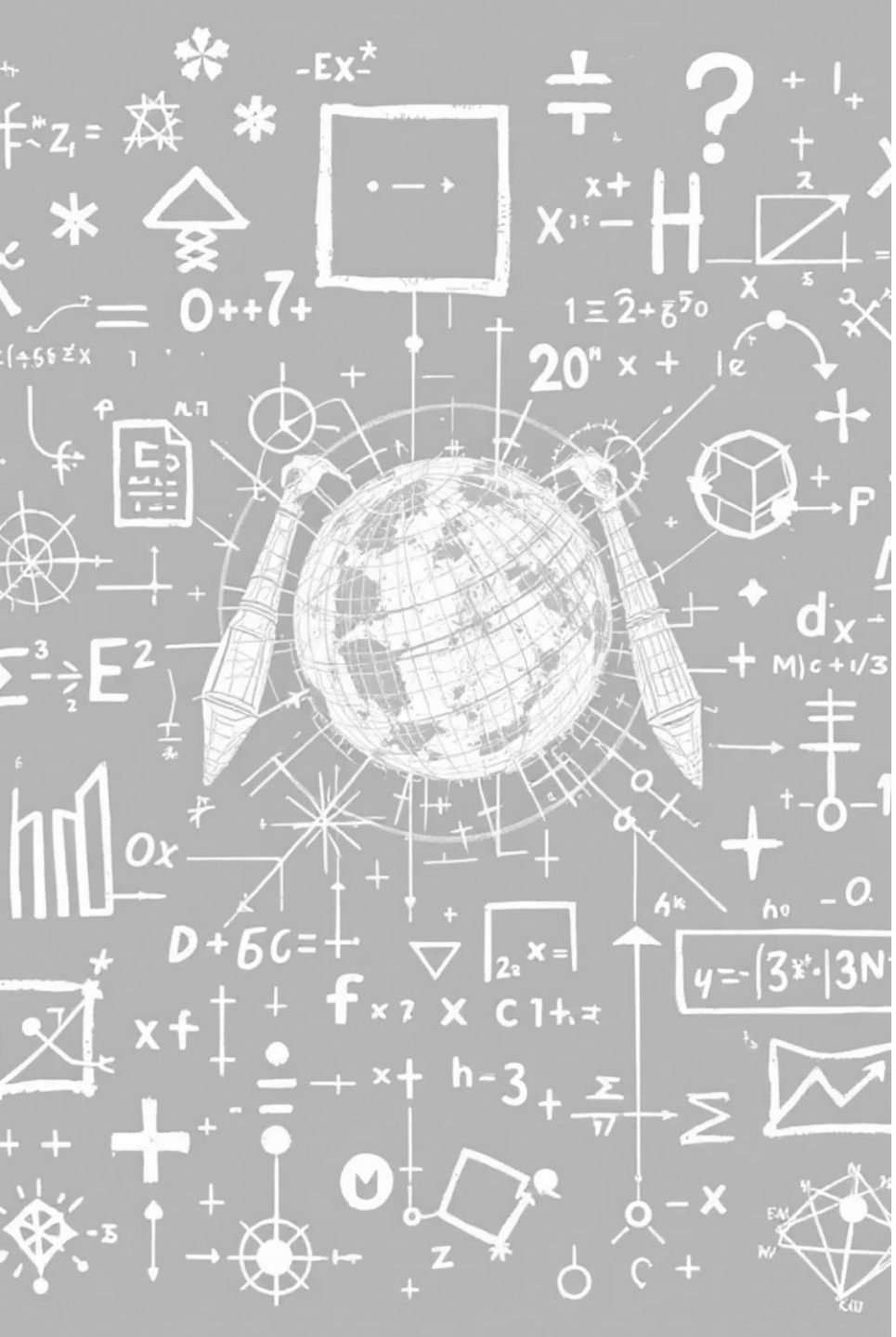
Лекция 1.
Введение в математическую логику

1.1. История развития математической логики

1.2. Аспекты математической логики

1.3. Логические антиномии (парадоксы)

1.4. Области применения



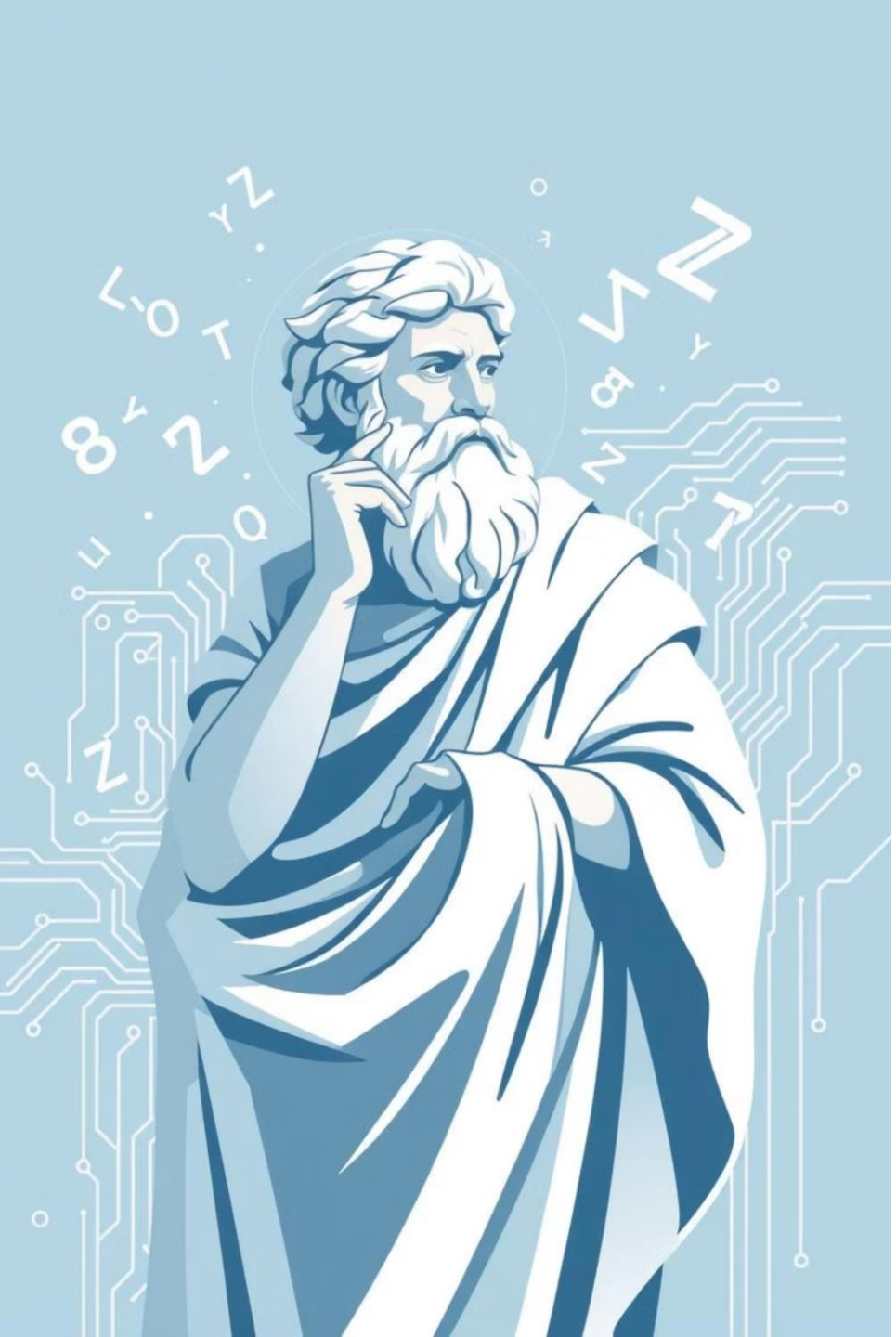
Лекция 1.
Введение в математическую логику

1.1. История развития математической логики

1.2. Аспекты математической логики

1.3. Логические антиномии (парадоксы)

1.4. Области применения



Математическая логика: путешествие сквозь века

Современная математическая логика — это наука о математических доказательствах. Ее история делится на два основных этапа, каждый из которых внес свой вклад в развитие этой фундаментальной дисциплины.



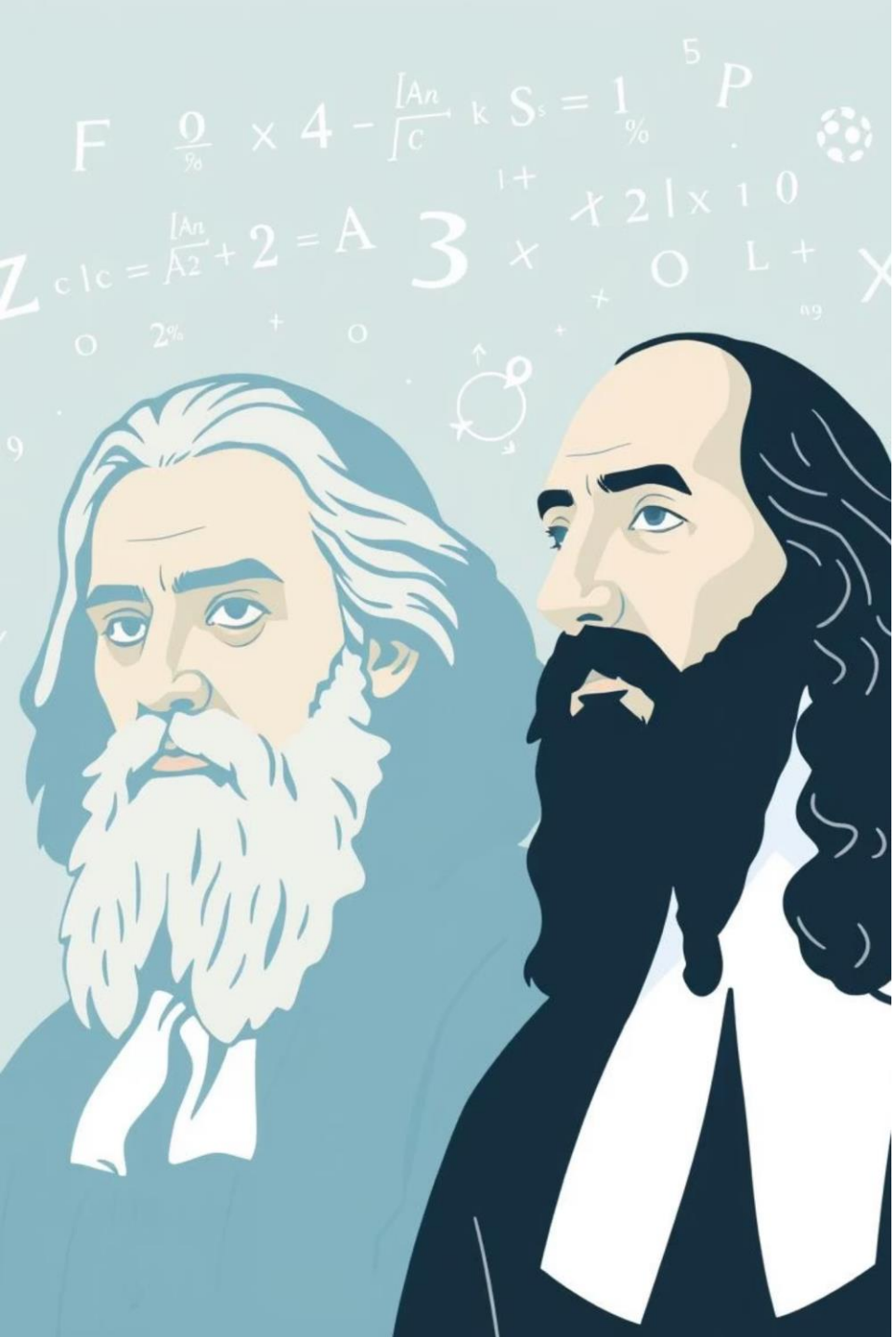
Истоки логики: Аристотель и Евклид

Аристотель (384–322 гг. до н. э.)

Основоположник логики как науки. В своих трудах, объединенных под общим названием «Органон», он выявил закономерности рассуждений, сформулировал основные законы логики и разработал теорию доказательств. Его работы содержат зачатки алгебры высказываний.

Евклид (330–275 гг. до н. э.)

Впервые предпринял попытку упорядочить геометрию с общелогических позиций, положив начало осознанию геометрии как аксиоматической теории.



Математизация логики: Лейбниц и Буль

1

Г. Лейбниц (1646–1716)

Положил начало применению математических методов в логике. Пытался построить универсальный язык, где «идеи заменялись бы вычислениями». Ввел буквенные обозначения для высказываний и сформулировал ряд законов алгебры высказываний.

2

Дж. Буль (1815–1864)

В работах «Математический анализ логики» и «Законы мысли» ввел операции сложения, умножения и отрицания высказываний. В его алгебре символы принимают значения 1 и 0 (истина и ложь), что стало основой для современной булевой алгебры.

Развитие булевой алгебры и аксиоматический метод

Совершенствование

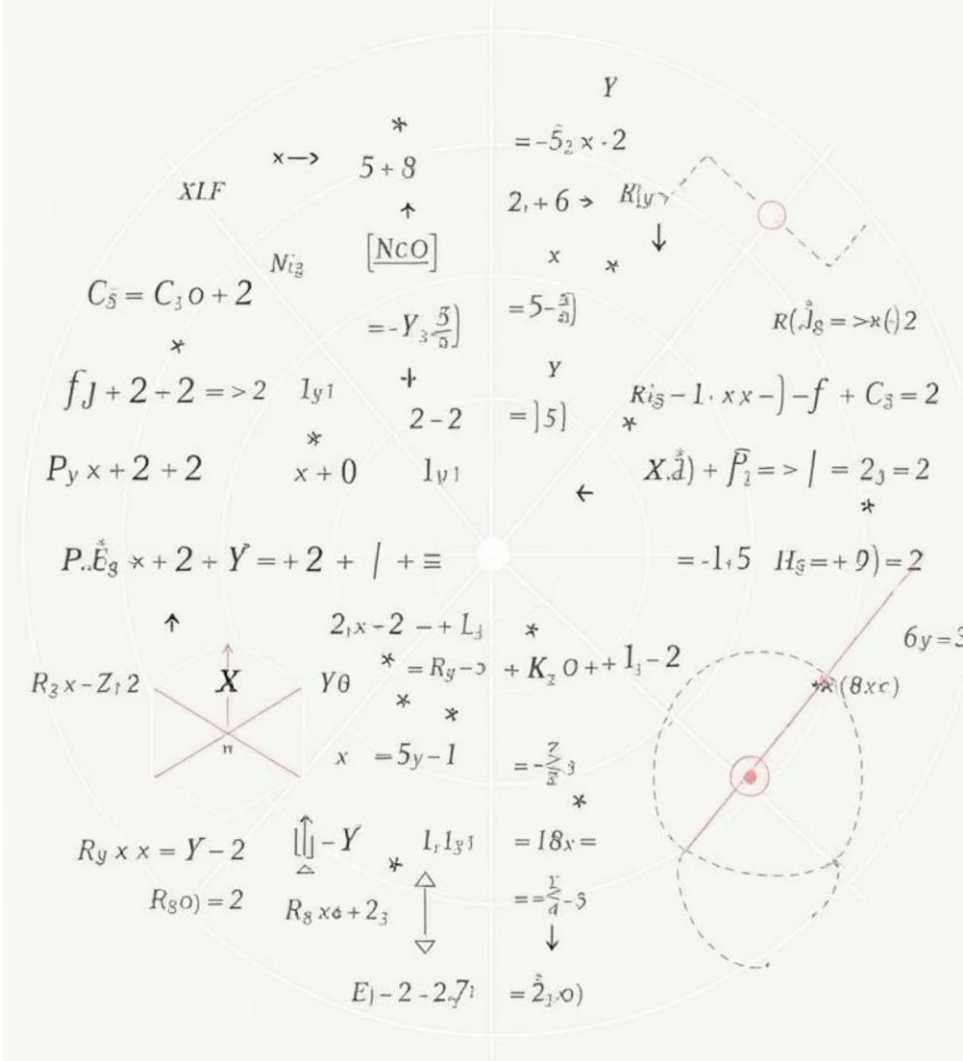
Алгебра Буля была доработана С. Джевансом (классификация булевых функций), Э. Шредером (принцип двойственности) и Г. Фреге (аксиоматический подход).

Вклад русских математиков

П.С. Порецкий (теория качественных умозаключений) и И.И. Жегалкин (алгебра высказываний как теория функций над полем из двух элементов) внесли значительный вклад.

Аксиоматический метод

К концу XIX века назрел новый этап развития математической логики, связанный с аксиоматическим методом и необходимостью доказательства непротиворечивости теорий.



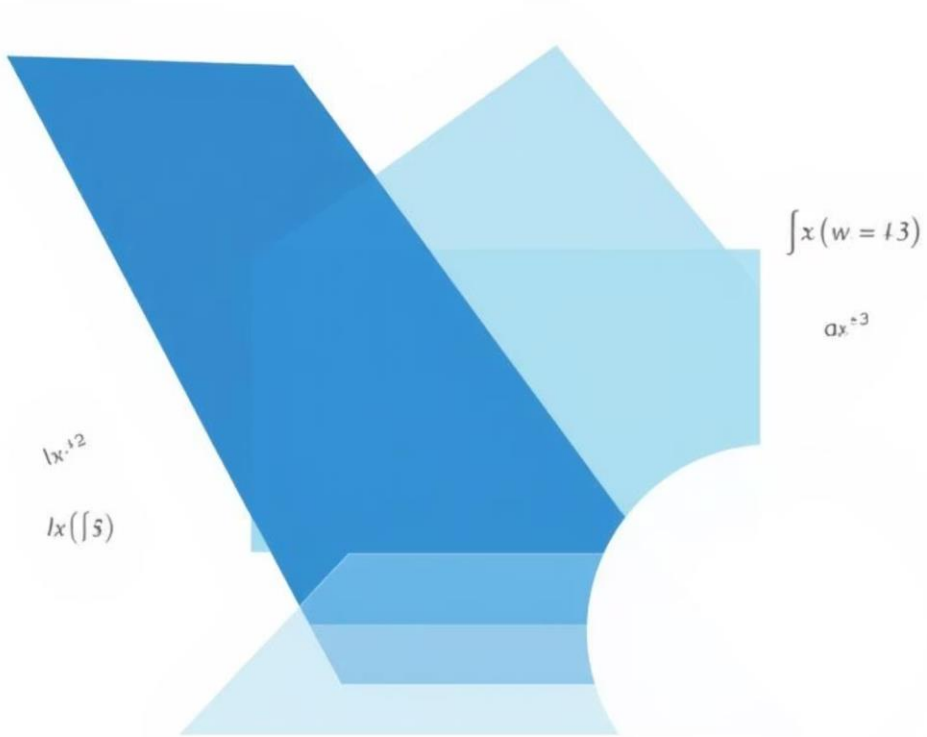
Проблема непротиворечивости и неевклидова геометрия

Вызов Лобачевского и Бойяи

Создание неевклидовой геометрии Н.И. Лобачевским и Я. Бойяи поставило вопрос о непротиворечивости аксиоматических теорий.

Метод интерпретаций

Ф. Клейн построил модель геометрии Лобачевского в рамках геометрии Евклида, а Д. Гильберт свел вопрос о непротиворечивости геометрий к арифметике.



Парадоксы и обоснование математики

Парадоксы в теории множеств конца XIX века показали, что трудности обоснования математики носят логический и методологический характер. Перед математической логикой встала задача исследования оснований математической науки.

Логицизм

Г. Фреге, Б. Рассел, А. Уайтхед стремились обосновать всю математику через логику.

Формализм

Д. Гильберт предложил записывать математические утверждения как логические формулы.

Интуиционизм

Л. Брауэр предложил отказаться от бесконечных множеств и закона исключенного третьего.





XX век: Бурное развитие и новые разделы

XX век стал периодом бурного развития математической логики, формирования многих ее новых разделов и

Теория множеств
Построены различные аксиоматические теории множеств.

Теория алгоритмов
Сформирована на базе математической логики, выработаны формализации понятия алгоритма.

Теория моделей
Возникла на стыке современной алгебры и логики.

Неклассические логики
Созданы многочисленные новые логические системы.

Вклад советских математиков

1

1925: А.Н. Колмогоров

Анализ закона исключенного третьего и интуиционизма.

2

1936: А.И. Мальцев

Локальная теорема, основы теории моделей, применение к теории групп.

3

1939: П.С. Новиков

Доказательство непротиворечивости арифметики.

4

1940-е: А.А. Марков

Теория нормальных алгоритмов, конструктивная математика.

5

1972: Ю.В. Матиясевич

Решение 10-й проблемы Гильберта (неразрешимость диофантовых уравнений).





Приложения математической логики

Математическая логика нашла широкое применение в различных областях, особенно в технике и информатике.

Электротехника

В.И. Шестаков применил алгебру высказываний для релейно-контактных схем (1935).

ЭВМ и программирование

Глубокое проникновение идей логики в конструирование ЭВМ и разработку программ.

Кибернетика

Влияние на развитие кибернетики, вычислительной математики и структурной лингвистики.

Ключевые выводы

1 **Два этапа развития**

От древнегреческих истоков до математизации и современных приложений.

2 **Фундаментальные концепции**

Аксиоматический метод, проблема непротиворечивости, алгебра высказываний.

3 **Вклад ученых**

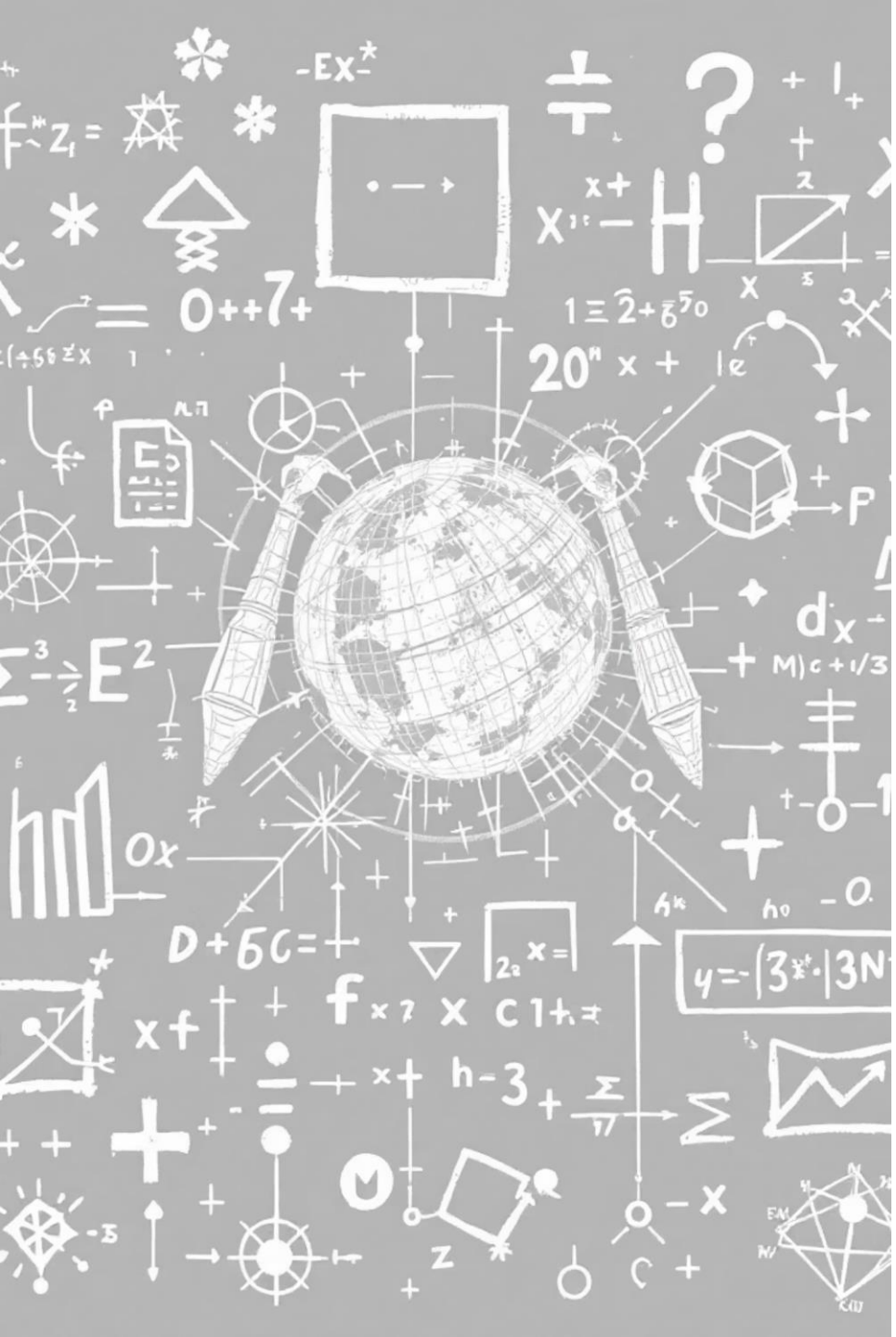
От Аристотеля и Лейбница до Буля и советских математиков.

4 **Широкие применения**

От электротехники до ИИ и программирования.

Математическая логика продолжает оставаться краеугольным камнем современной науки и технологий.





Лекция 1.
Введение в математическую логику

1.1. История развития математической логики

1.2. Аспекты математической логики

1.3. Логические антиномии (парадоксы)

1.4. Области применения





Математическая логика: Мост между философией и математикой

Суть математической логики

Математическая логика — это дисциплина, которая исследует формы мышления и методы вывода, используя абстрактный математический аппарат. Она стала первым проникновением математики в гуманитарную область, в сферу человеческого мышления.

Логика по предмету

Изучает мышление и его законы.

Математика по методу

Использует строгие математические инструменты.



Теорема Гёделя о неполноте

Доказанная Куртом Гёделем в 1931 году теорема о неполноте формальной арифметики показала, что математическая логика имеет самое прямое отношение к человеческому мышлению.

- Невозможно создать полную и непротиворечивую систему аксиом для арифметики.
- Аксиоматический подход не охватывает все истинные арифметические утверждения.



Логика как инструмент педагогики математики

В процессе обучения математике логика выступает как инструмент изучения математики. Это особый инструмент, который помогает более осознанно и глубоко изучать саму математику.

Инструмент

Помогает в изучении математики.

Объект изучения

Изучается в рамках математики.



Принципы обучения математике с помощью логики

1 **Строение математических утверждений**

Умение видеть логическую структуру определений и теорем, выявлять логические связки.

2 **Понятие доказательства**

Понимание доказательства как последовательности утверждений, выводимых по правилам.

3 **Методы доказательства**

Освоение синтетического, аналитического методов и доказательства от противного.



Принцип обучения строению математических утверждений

Важно научиться видеть логическую структуру математического утверждения, будь то определение или теорема, и отчетливо видеть, где и какие логические связки участвуют в формулировке.

Определения

Типы: через род и вид, индуктивные, аксиоматические.

Теоремы

Что дано, что доказать, структура условий и заключения.

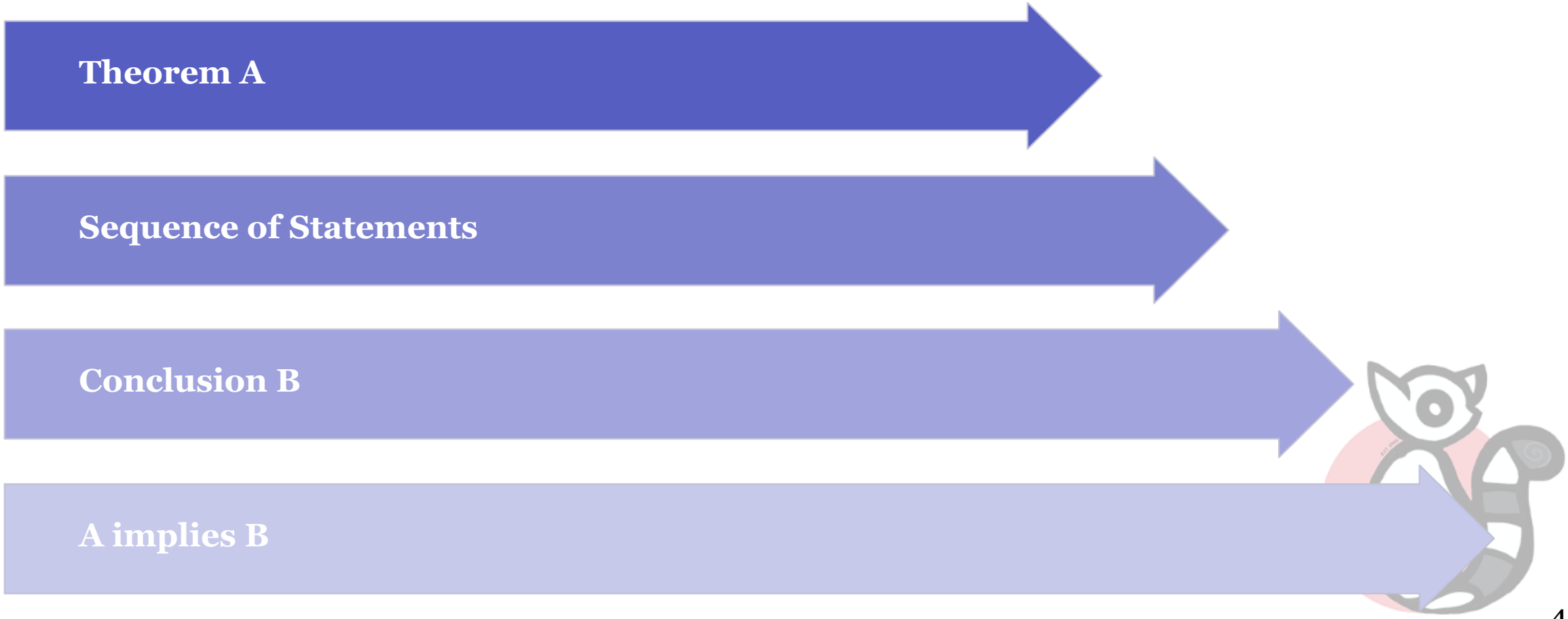
Эквивалентность

Преобразование утверждений равносильным образом.



Принцип обучения понятию доказательства

Доказательство теоремы — это последовательность утверждений, каждое из которых является либо условием теоремы, либо аксиомой, либо получено из предыдущих утверждений по правилу вывода.



Принцип обучения методам доказательства

Освоение методов построения цепочки утверждений для доказательства теоремы A→B.

Синтетический (прямой) метод

Построение цепочки от A к B.

Аналитический метод

Построение цепочки от B к A.

Также важно освоить методы доказательства от противного и приведения к абсурду.



Принцип обучения строению математических теорий

Уяснение сути аксиоматического метода при построении и преподавании математической теории, а также понимание первоначальных понятий, аксиом и теорем.

- **Аксиоматический метод:** понимание его применения в геометрии (Евклид, Гильберт) и алгебре.
- **Метатеория:** свойства теории (непротиворечивость, полнота, категоричность).



Математическая логика и современные ЭВМ

Математическая логика является теоретической основой алгоритмического мышления и играет ключевую роль в развитии компьютерных технологий.



Конструирование

Алгебра высказываний для создания элементной базы компьютеров.



Математическое обеспечение

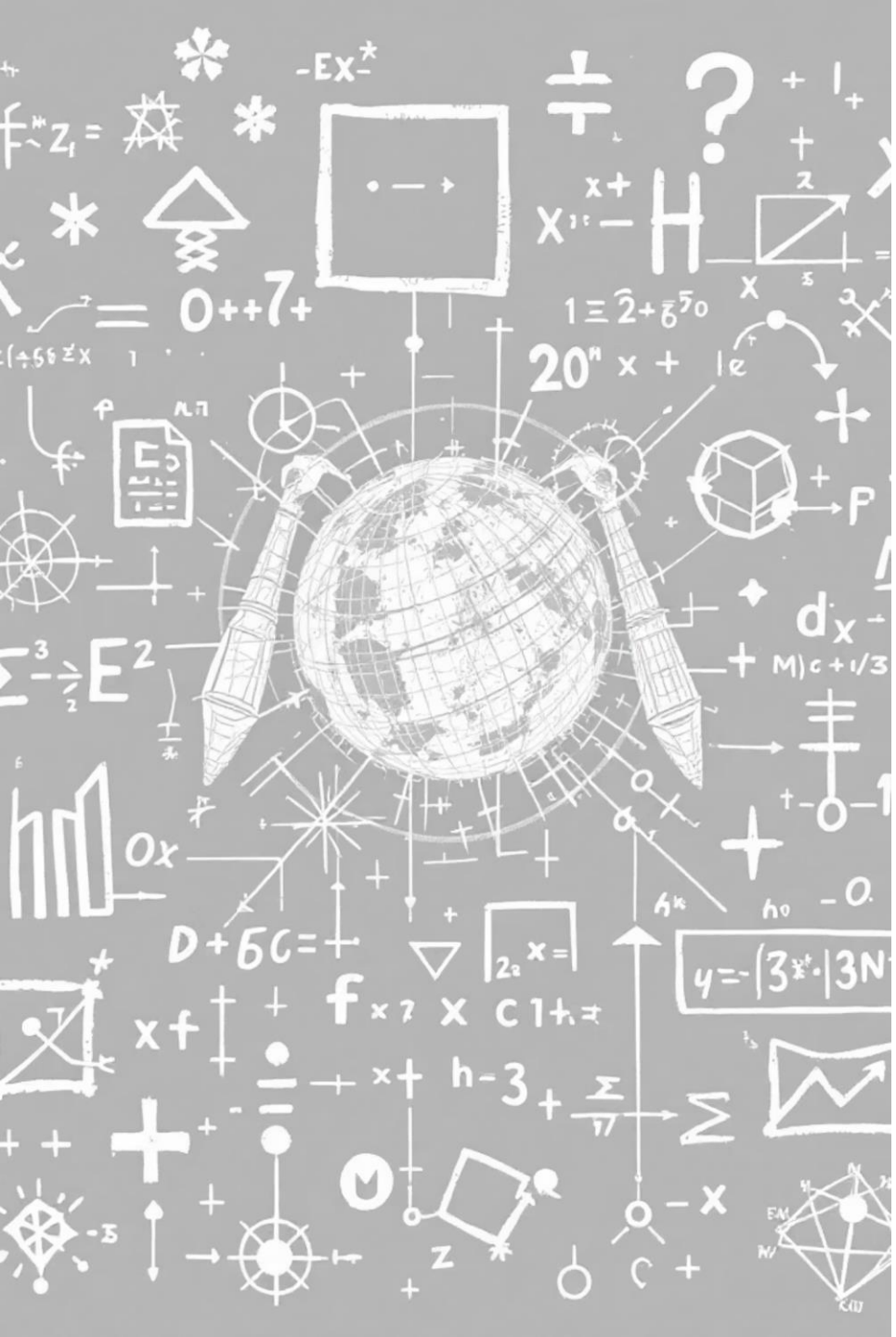
Теория алгоритмов и формальных систем для языков программирования.



Искусственный интеллект

Синтез логики и компьютеров для баз данных и экспертных систем.





Лекция 1.
Введение в математическую логику

1.1. История развития математической логики

1.2. Аспекты математической логики

1.3. Логические антиномии (парадоксы)

1.4. Области применения



Математическая логика: Преодолевая Противоречия

Математическая логика помогает преодолевать противоречия, возникающие из логического анализа. Реальный мир полон противоречий, которые отражаются в наших рассуждениях, вызывая недоумение или смех.



Понятия

Парадокс

Приближенно к истине, но не истина (как бы истина).

Антиномия

Противоположно истине.

Парадокс — это «попадание в молоко» при стрельбе в мишень, тогда как антиномия — это стрельба в инструктора вместо мишени.



Парадокс Кучи

Одна картошка — не куча, две — тоже. Добавление одной картофелины не делает не-кучу кучей. Следовательно, куч картошки не бывает.

Аналогично: куч зерна, песка не бывает. Формально рассуждение кажется правильным, ведь последовательным прибавлением единицы можно получить любое число.





Парадокс лжеца

> Некто (лжец) говорит: «Я лгу».

Если он говорит правду, то он лжет, что является ложью. Если он лжет, то он говорит правду, что является истиной. Оба предположения приводят к противоречию, делая высказывание неопределимым.



Парадокс критянина Эпименида

Эпименид, критянин, утверждал, что все утверждения, сделанные критянами, ложны. Этот парадокс произвел огромное впечатление на греков, и, по легенде, даже привел к самоубийству некоего Филита Косского.

Древнеегипетский парадокс

В Древнем Египте «на север» означало «туда, куда течет река» (Нил), а «на юг» — «туда, откуда течет река». Когда египетские послы из Вавилона писали, что Тигр и Евфрат текут «туда, откуда реки текут» и «туда, куда реки текут», писцам приходилось ломать голову.

Этот пример показывает, что источником парадоксов может быть неразвитость языка.



Парадокс брадобрея

В гарнизоне брадобрей должен брить тех и только тех, кто не бреется сам. Вопрос: кто бреет самого брадобрея?

- Если он бреется сам, то он бреет того, кто бреется сам (нарушение приказа).
- Если он не бреется сам, то он не бреет того, кто не бреется сам (нарушение приказа).



Этот парадокс демонстрирует противоречия, которые могут возникать в правилах и законах.



Современные аналогии

Парадокс брадобрея актуален и сегодня. Представьте указание: «Дежурный проводник наливает чай тем и только тем, кто не наливает себе его сам». Как быть с чаем самому проводнику?

Противоречия в приказах и законах недопустимы, но некоторые страны десятилетиями живут с противоречивым законодательством, что приводит к страданиям, но позволяет немногим преуспевать.

Парадокс Рассела

Рассмотрим множество всех таких множеств, которые не являются элементами самих себя. Например, множество всех котов не является элементом самого себя.

Если это множество является элементом самого себя, то оно не является элементом самого себя. Если оно не является элементом самого себя, то оно является элементом самого себя. Это приводит к противоречию.



Другие известные парадоксы

Парадокс Греллинга

Автологично ли прилагательное «гетерологичный»?

Парадокс всемогущества

Может ли всемогущее Существо создать камень, который не сможет поднять?

Эти и другие парадоксы, такие как парадокс Бурали-Форти, Кантора, Сколема и Ришара, продолжают стимулировать глубокие размышления в математической логике.



Парадоксы и софизмы

исследуем увлекательный мир парадоксов и софизмов, разберем их природу, примеры и методы их разрешения.



Природа парадоксов

Парадоксы часто возникают из-за самоотносительности, когда смысл утверждения (его семантика) противоречит системе знаков (семиотике) или правилам языка (синтаксису), на котором оно сформулировано.

Семантика

Смысл утверждения

Семиотика

Система знаков

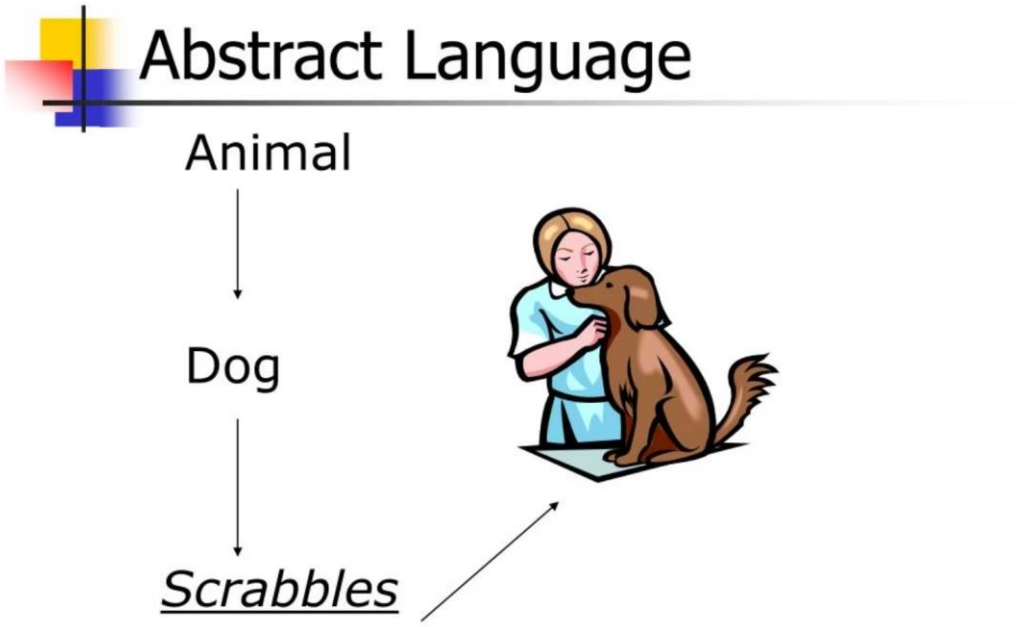
Синтаксис

Правила языка



Метаязык как решение

Один из способов борьбы с парадоксами самоотносительности — использование метаязыка. Это язык, на котором мы говорим о данном языке, с запретом говорить о языке на самом этом языке.



Пример: нотация Бэкуса

Формальный язык для описания формальных языков программирования является метаязыком по отношению к языкам программирования.



Парадоксы в повседневной жизни

Парадоксы самоотносительности не так далеки от наших реалий, как может показаться. Они встречаются в советах, высказываниях и даже в юморе.

Советы

«Послушай моего совета – не слушай ничьих советов!»

Высказывания

«Все, что написано курсивом – чепуха»

Юмор

– А Вы культурный человек? – Ага! (ответ на вопрос о культурном человеке и слове "ага")



Различение имен и объектов

Один из способов борьбы с парадоксами — четкое различение имен объектов и самих объектов. Имя используется как объект некоторого языка, а денотат (объект, обозначаемый именем) находится вне этого языка.



> Преподаватель пишет уравнение «$x^2 - 3x + 2 = 0$» и говорит студенту: «найдите x».
> Тот тычет пальцем в букву «x» и радостно восклицает: «Так вот же он!».



Парадокс против Софизма

Важно различать парадоксы и софизмы. Парадокс — это рассуждение, которое приводит к противоречию, используя логически допустимые средства. Софизм — это мнимое доказательство, основанное на преднамеренном нарушении логических правил.

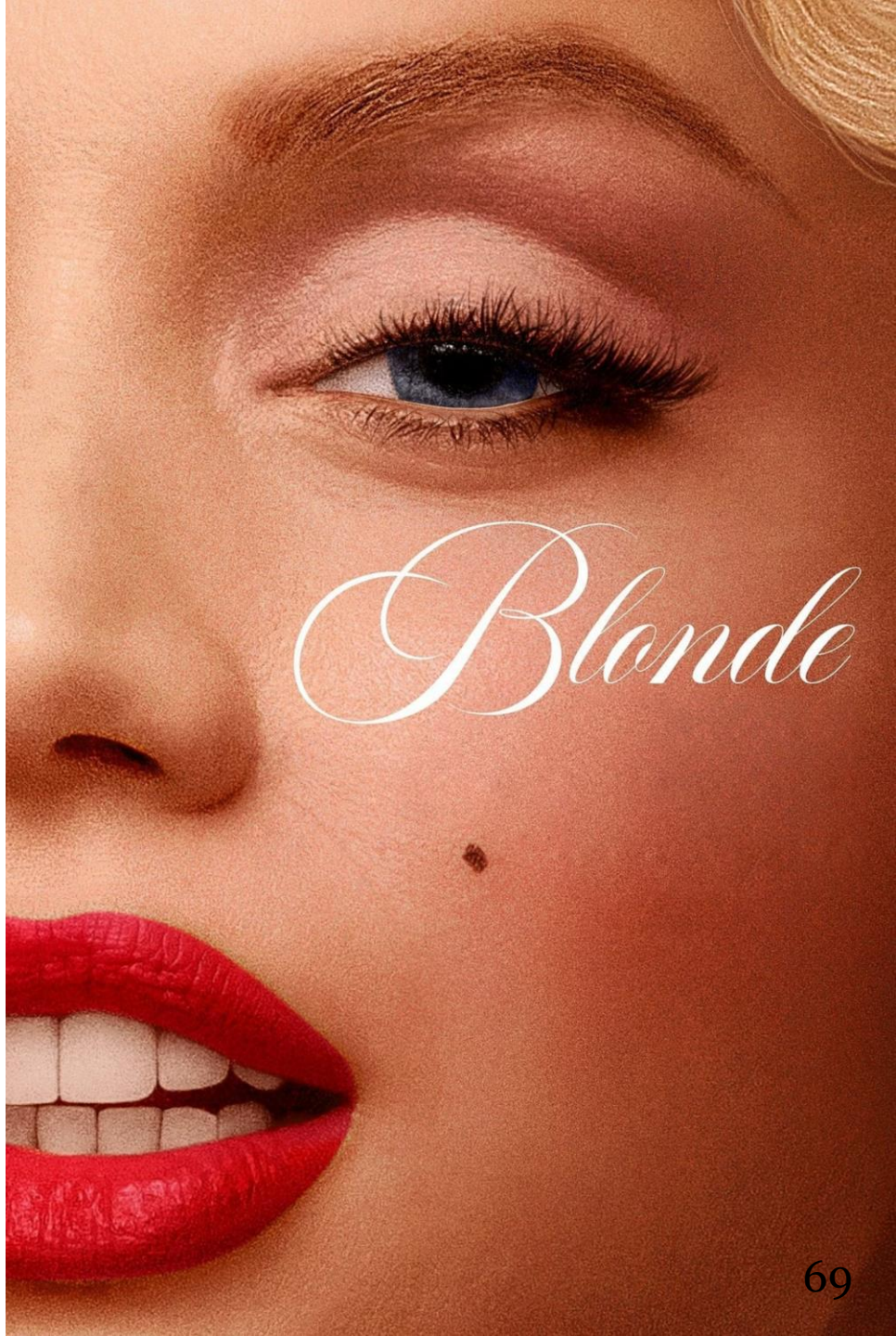
Парадокс	Софизм
Непреднамеренное противоречие в логике.	Преднамеренное нарушение логических правил.



Задача от блондинки: пример логической ошибки

Задача от блондинки — это пример логической ошибки, возникающей непроизвольно из-за невысокой логической культуры.

> Я взяла у друга в долг 100 рублей, которые благополучно потеряла. Тогда я пошла к подруге и взяла в долг еще 50 рублей. Купила две шоколадки по 10 рублей. Оставшиеся 30 рублей вернула другу. Осталась должна 70 рублей другу и 50 рублей подруге, всего 120 рублей. Плюс купленные шоколадки. Итого 140 рублей. Куда подевались 10 рублей?



Софизмы: искусство логических ловушек

Добро пожаловать в мир софизмов — утверждений, которые кажутся логичными, но содержат скрытую ошибку. Сегодня мы рассмотрим несколько увлекательных примеров из разных областей.



Древнегреческие софисты

Софистами называли группу древнегреческих философов IV–V века до н. э., которые достигли большого искусства в логике. Их целью было научить убедительно защищать любую точку зрения, допуская логические передержки.



- Учителя красноречия
- Приобретение и распространение мудрости
- Защита любой точки зрения
- Допускались логические передержки

Известные софизмы

Существует множество софизмов, созданных еще в древности, заключения которых носят курьезный характер.

Софизм «вор»

«Вор не желает приобрести ничего дурного: приобретение хорошего есть дело хорошее: следовательно, вор желает хорошего».

Софизм «лекарство»

«Лекарство, принимаемое больным, есть добро; чем больше делать добра, тем лучше; значит, лекарство нужно принимать в больших дозах».

Софизм «рогатый»

«То, что ты не потерял, ты имеешь; ты не потерял рога, следовательно, ты их имеешь».

Софизм Эватла и Протагора

> «Судьи или присудят тебя к уплате гонорара или не присудят. В обоих случаях ты должен будешь уплатить. В первом случае в силу приговора судьи, во втором случае в силу нашего договора».

Эватл, ученик софиста Протагора, договорился об оплате только после выигрыша первого процесса. Протагор требовал гонорар, утверждая, что Эватл заплатит либо по решению суда, либо по договору. Эватл же отвечал, что не заплатит ни в одном случае: если проиграет, то по договору, если выиграет, то по решению суда.





Софизм о вечном двигателе

Этот софизм, связанный с физикой, утверждает возможность вечного двигателя четвертого рода.

> «Вечный двигатель первого рода невозможен, поскольку его запрещает первое начало термодинамики; вечный двигатель второго и третьего рода запрещают соответственно второе и третье начала термодинамики; поскольку четвертого начала термодинамики нет, то вечный двигатель четвертого рода возможен».

Это игра слов, основанная на отсутствии четвертого закона термодинамики, что не означает его возможности.



Математические софизмы

Математика, казалось бы, точная наука, но и здесь встречаются софизмы, основанные на неверных преобразованиях или скрытых допущениях.

Тождественные преобразования

Использование операций со степенями и мнимой единицей может "доказать", что $-1 = 1$.

Квадратное уравнение с тремя корнями

Уравнение $(x-a)(x-b)(x-c) = 0$ может быть представлено как квадратное, но имеет три корня.

Софизм: квадратное уравнение с тремя корнями

Обычно квадратное уравнение имеет не более двух корней. Однако, рассмотрим уравнение:

$$\frac{(x-a)(x-b)}{c-a} + \frac{(x-b)(x-c)}{a-b} + \frac{(x-c)(x-a)}{b-c} = 0$$

Здесь a, b, c – любые различные числа. Подставив $x=a$, $x=b$ или $x=c$, мы увидим, что уравнение обращается в верное равенство. Таким образом, это квадратное уравнение имеет три различных корня.



Софизм с числами Фибоначчи

Этот софизм демонстрирует, как перекомпоновка фигур может "изменить" площадь.

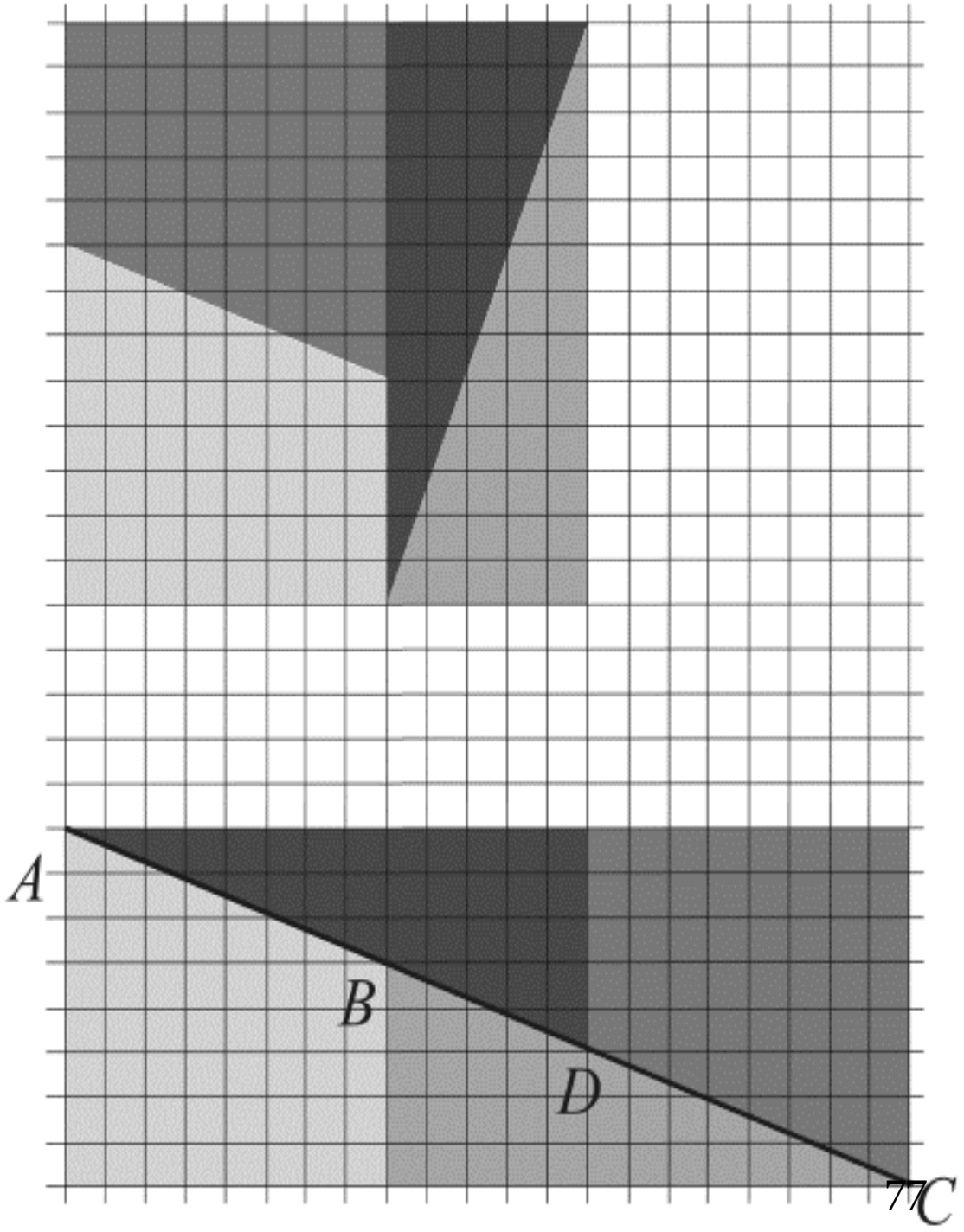
Пример 1: Сторона 13

Квадрат со стороной 13 (площадь 169) разрезается на 4 части и составляется в прямоугольник со сторонами 8 и 21 (площадь 168).

Пример 2: Сторона 21

Квадрат со стороной 21 (площадь 441) разрезается на 4 части и составляется в прямоугольник со сторонами 13 и 34 (площадь 442).

Разница в площади возникает из-за небольшой щели или наложения, незаметного глазу. Аналогичное построение возможно для любых трех последовательных чисел Фибоначчи.

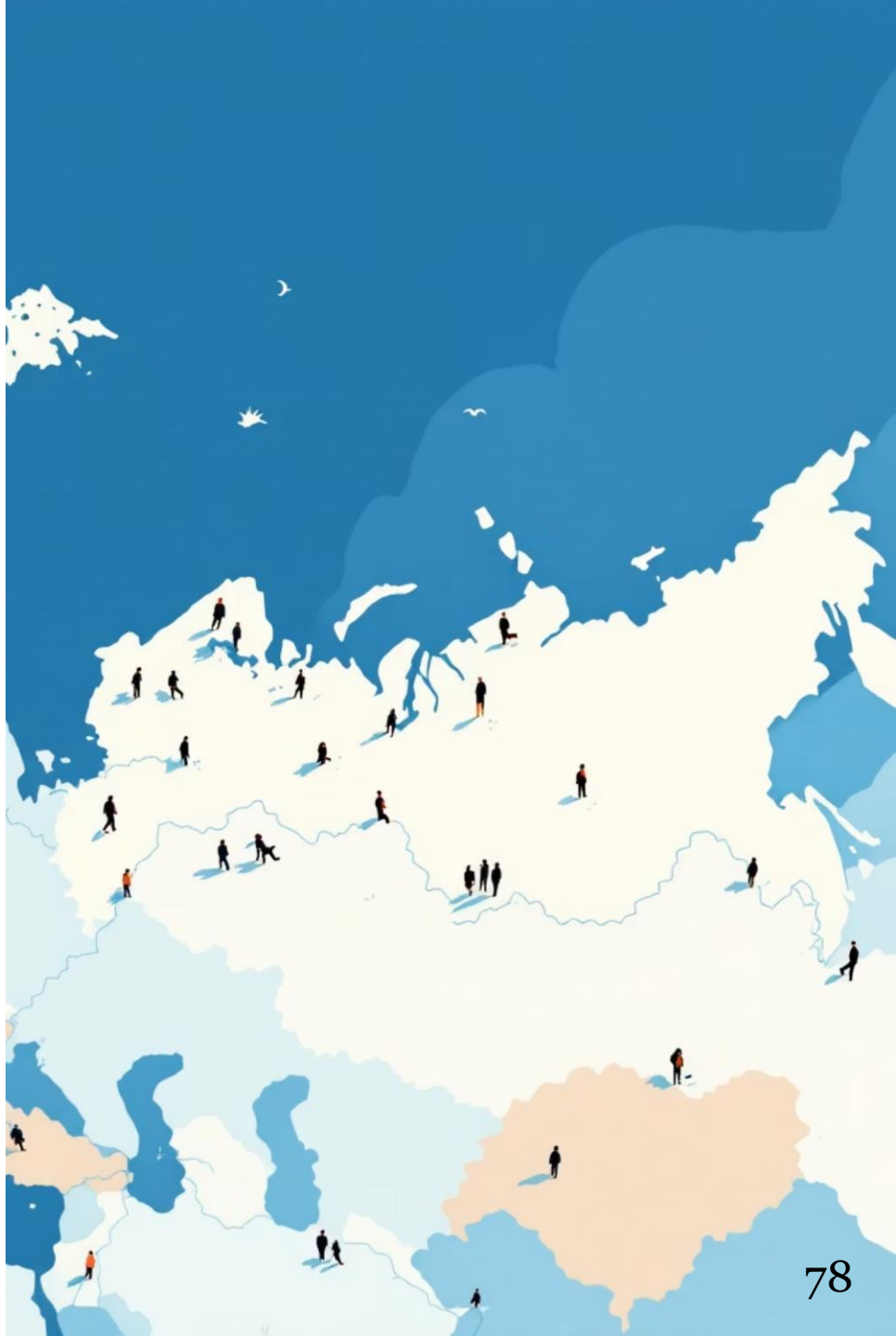


Софизм: карта России

Этот софизм играет с масштабом и пропорциями, создавая абсурдное заключение.

> «На карте России масштаба 1:15 000 000 все размеры уменьшены в 15 миллионов раз. Если население в 150 миллионов уменьшить во столько же раз – останется 10 человек. Казалось бы, им должно хватить места. Но на карте и одному тесно».

Софизм возникает из-за некорректного применения масштаба к несвязанным величинам (площадь и количество людей).





Ключевые выводы

1 **Логические ошибки**

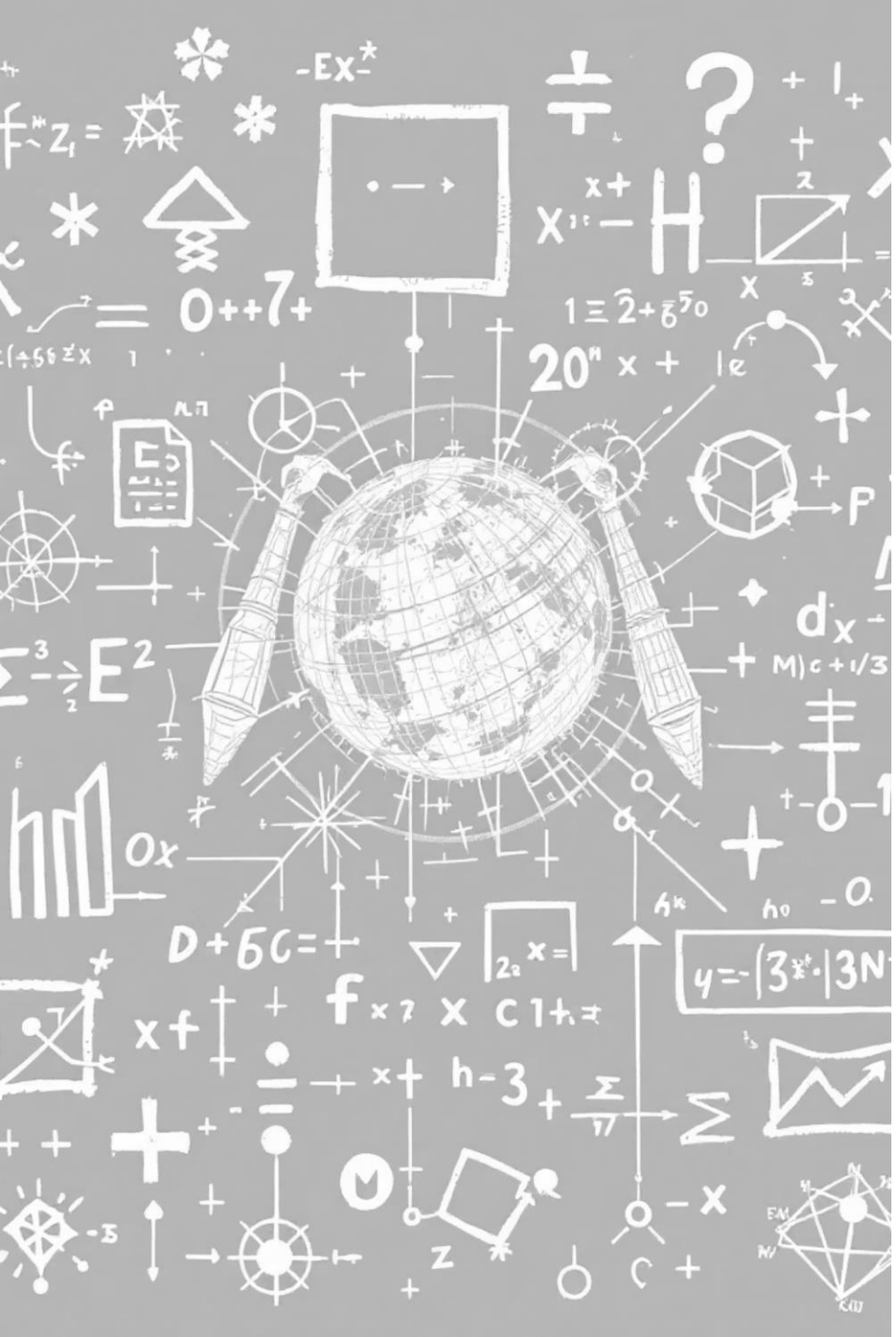
Софизмы часто скрывают ошибки в рассуждениях, играя на нашем восприятии.

2 **Внимательность к деталям**

Важно тщательно проверять предпосылки и выводы, особенно в математике и физике.

3 **Искусство убеждения**

Софизмы показывают, как можно использовать логику для создания иллюзий или манипуляции.



Лекция 1. Введение в математическую логику

1.1. История развития математической логики

1.2. Аспекты математической логики

1.3. Логические антиномии (парадоксы)

1.4. Области применения

Математическая логика и вычислительные науки

Исследование роли математической логики в формировании и развитии вычислительных наук.



Что такое вычислительные науки?

«Вычислительные науки» — это область научного знания, изучающая вычислительную технику, программное и алгоритмическое обеспечение, а также применения ЭВМ.

Физика

Химия

Математика

Лингвистика

Биология

Психология

Они включают разделы различных естественных и общественных наук, выступающих как потребителями, так и поставщиками результатов.

Математика в вычислительных науках

Важное место занимает часть математики, называемая теоретической информатикой, вычислительной теорией, алгоритмикой, кибернетикой, или «computer science» за рубежом.



Разделы математики

- Математическая физика
- Теория численных методов
- Теория массового обслуживания
- Теория игр

Уникальность математической логики

Математическая логика — единственная математическая дисциплина, изучающая взаимоотношение между текстами и их смыслом.

Тексты и смысл

Исследование этого взаимоотношения становится первостепенным, когда тексты превращаются в средство взаимодействия с компьютером.

Теория алгоритмов

Изучает повелительные тексты (программы), содержание которых составляет алгоритм.

Связь теории алгоритмов с вычислительной практикой очевидна.



Роль математической логики в программировании

С развитием программирования роль математической логики в узком смысле слова становится все более важной.

Переход к новому программированию
От явного, «процедурного», «повелительного» описания все чаще переходят к «непроцедурному», «функциональному», «повествовательному» программированию.



Влияние на вычислительную практику

Воздействие математической логики на вычислительную практику проявляется в нескольких аспектах:

01

Формирование методологии

Создание общей методологии построения аппаратного и программного обеспечения.

02

Выбор языков и моделей

Влияние на выбор языков и вычислительных моделей.

03

Конкретные теоремы и алгоритмы

Использование конкретных теорем и алгоритмов математической логики.



Дж. фон Нейман и А. Тьюринг внесли фундаментальный вклад в обе области.

Открытие вычислимой функции

Одно из важнейших достижений математики XX века — открытие общего понятия вычислимой функции, то есть функции, вычисляемой каким-либо алгоритмом.

Это открытие тесно связано с тезисом Чёрча и идеей универсального программирования.



Тезис Чёрча

Тезис Чёрча утверждает, что любой язык программирования, использующий потенциально неограниченные ресурсы, будет универсальным.

1

Универсальность языка

Пригоден для записи программы вычисления любой вычислимой функции.

2

Простота и эффективность

Небольшое семейство простейших функций и алгоритмических конструкций дает язык для произвольных функций.

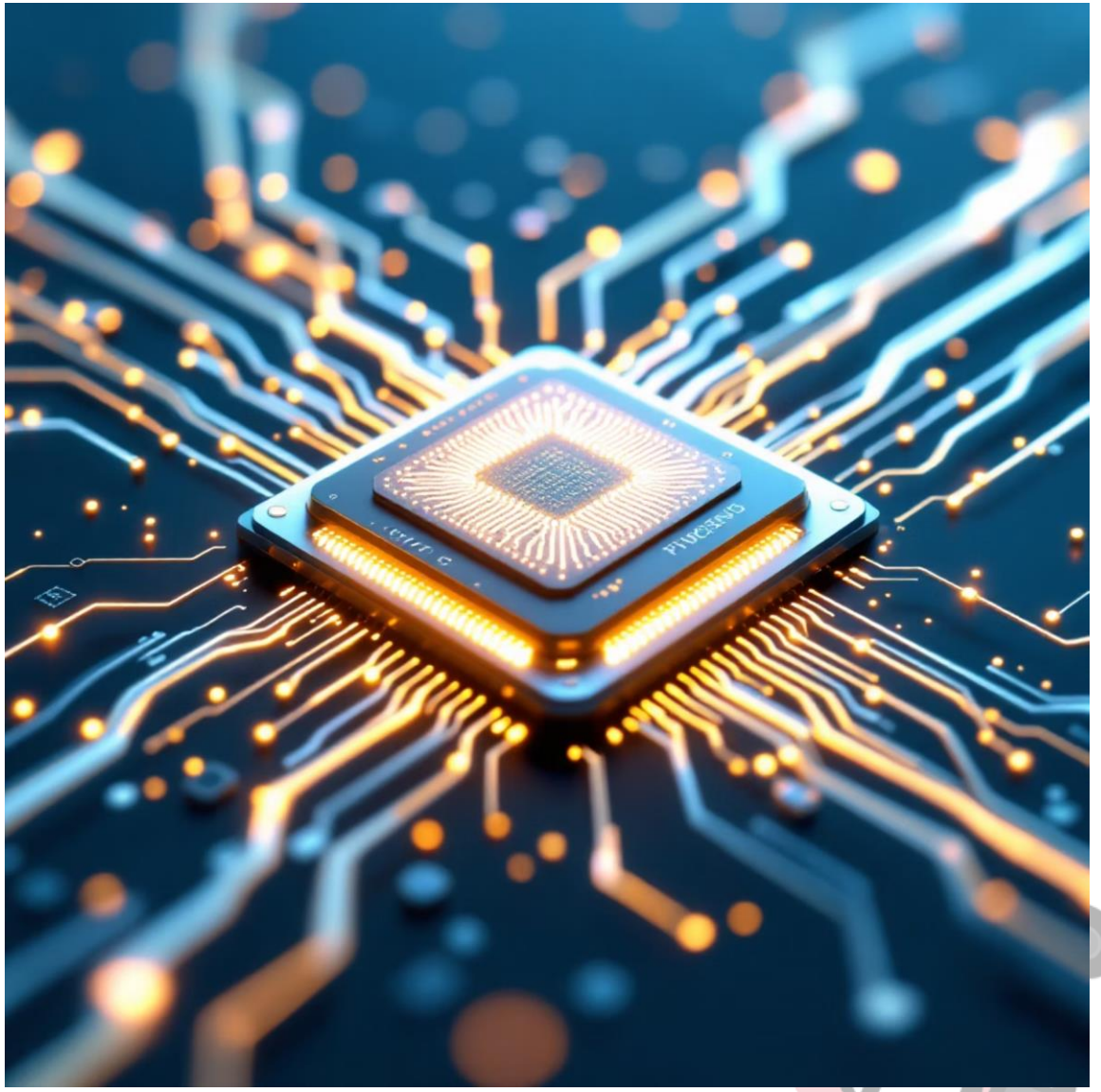
Это позволяет создавать универсальные машины вместо специализированных устройств.



Второе рождение идеи

Идея универсальной машины, выдвинутая Э. Л. Постом, А. М. Тьюрингом и А. Чёрчем в 30-е годы, пережила второе рождение.

Применение микропроцессоров
Связано с использованием универсальных микропроцессоров, заменивших разработку специализированных устройств автоматики.



Ключевые выводы

Фундаментальная роль

Математическая логика является фундаментом вычислительных наук, особенно взаимодействия человека с компьютером.

Универсальность

Тезис Чёрча и идея универсального программирования стали ключевыми для развития ЭВМ.

Будущее программирования

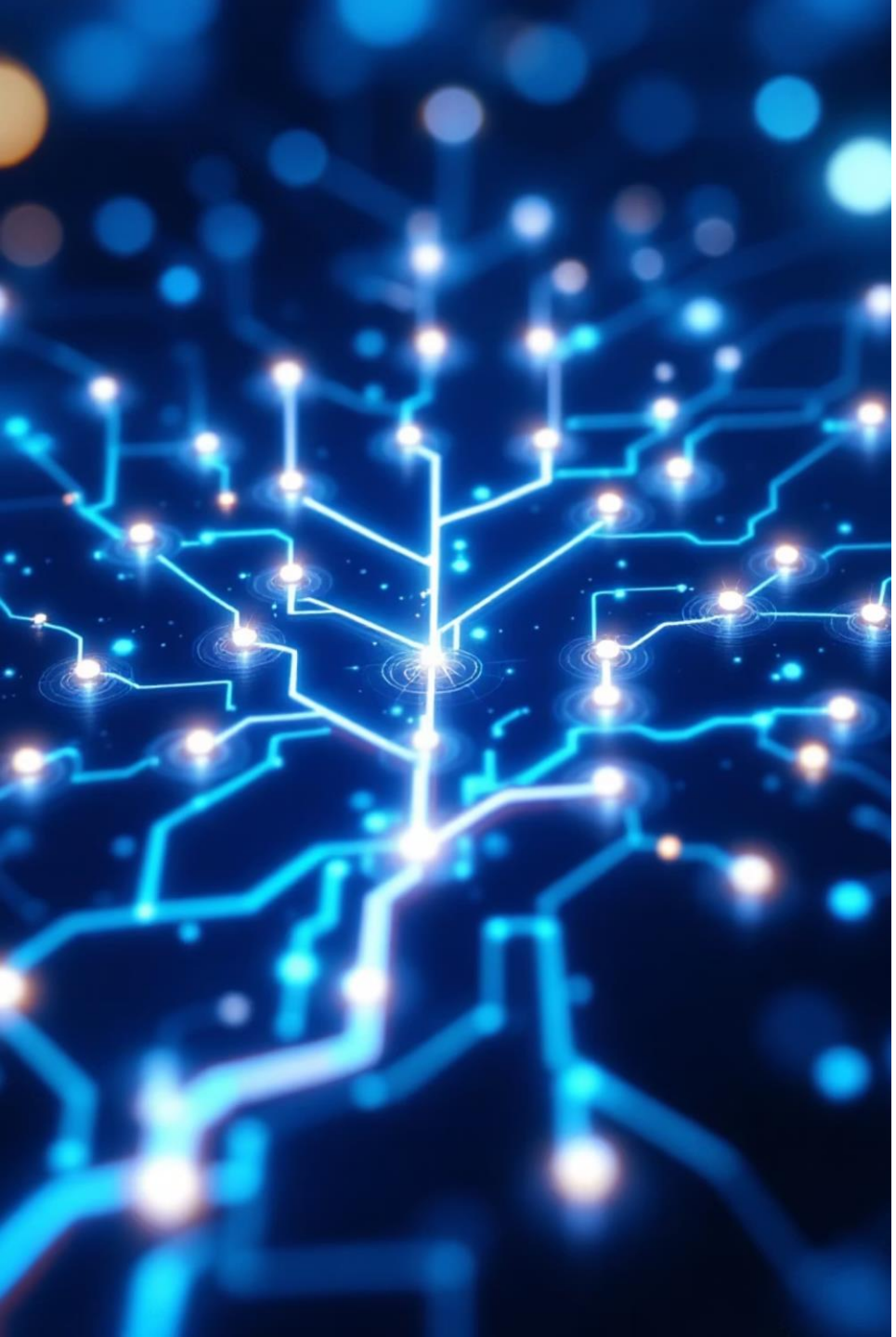
Переход к «повествовательному» программированию подчеркивает растущую важность математической логики.

Эти концепции продолжают формировать современную вычислительную практику.



От алгоритмов к исчислениям: путешествие в мир формальных систем





Разрешающий тип языков

Естественные языки допускают конструкции «разрешающего типа», позволяющие, а не предписывающие, выполнять те или иные действия. Выбор действия зависит от воли участников, случая и т.д. Так возникают понятия исчисления, вероятностного алгоритма, игры.



Понятие исчисления и породимые множества

Как конкретному алгоритму соответствует вычисляемая функция, так конкретному исчислению соответствует порождаемое им множество. Общему понятию исчисления соответствует общее понятие породимого множества.

1

Пример 1

Множество всех верных равенств между неопределенными интегралами.

2

Пример 2

Множество всех выражений формального языка (мат. логики, программирования), построенных по правилам.





Тезис Поста и вычислимость

Аналогично тезису Чёрча, тезис Поста утверждает совпадение интуитивного понятия породимости с понятием породимости универсальным исчислением. Породимые множества — это в точности множества значений (и области определения) вычислимых функций.

Однако, среди породимых множеств есть такие, для которых невозможен алгоритм распознавания принадлежности. Например, нет алгоритма проверки, зацикливается ли программа.





Революция в программировании: отделение текста от смысла

Идея отделения текста на языке от его смысла и возможность сопоставления с текстом различных смыслов является революционной для программирования. Формальные определения семантики языков математической логики послужили образцом для многих построений в программировании.



Взаимодействие семантик и доказательство правильности программ

Тонкое взаимодействие между повествовательной (функциональной) и повелительной (операционной) семантиками важно для достижения большей надежности программного обеспечения. С этим связана область доказательства правильности программ.

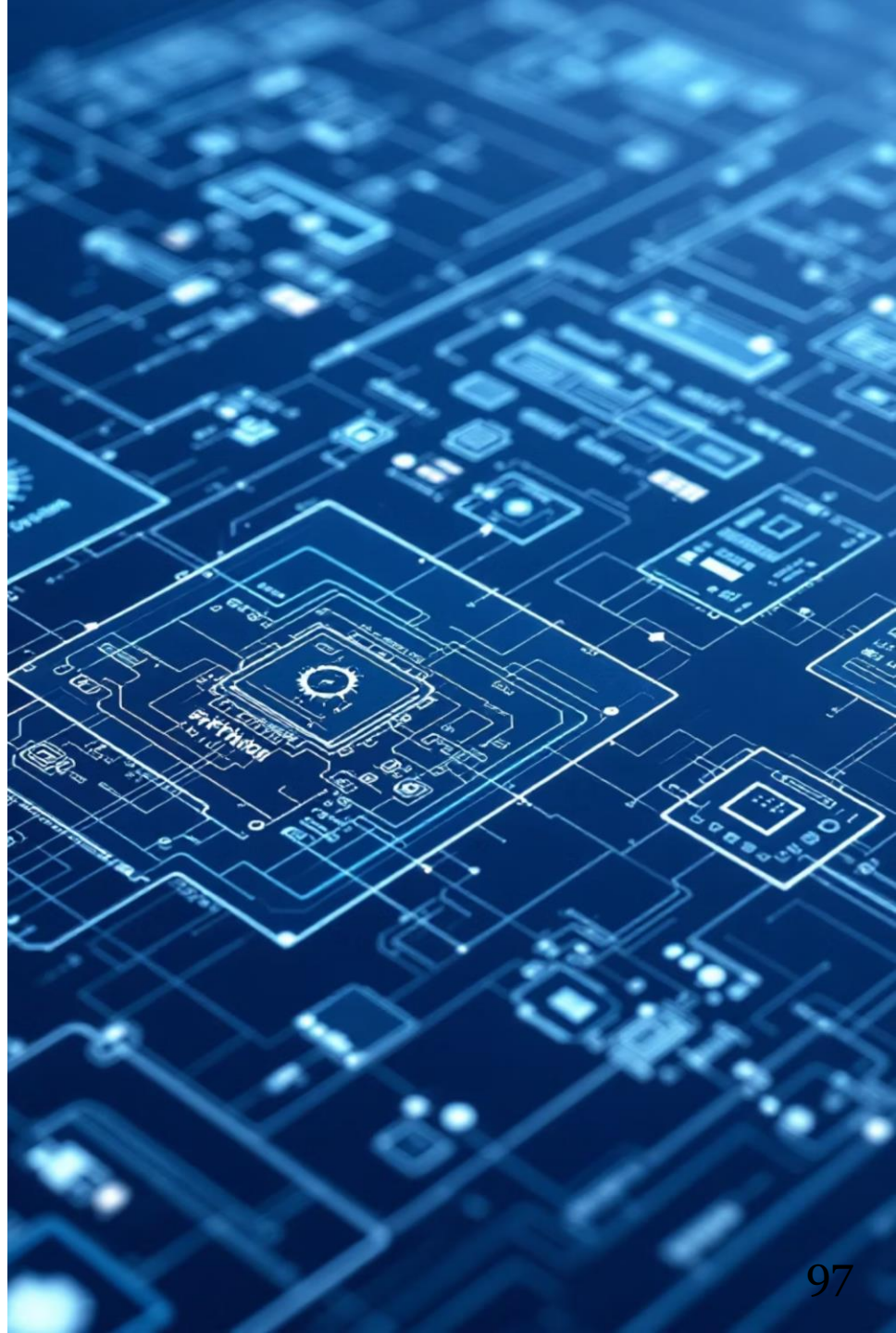
Сама постановка задачи доказательства правильности была бы невозможна без достижений математической логики.

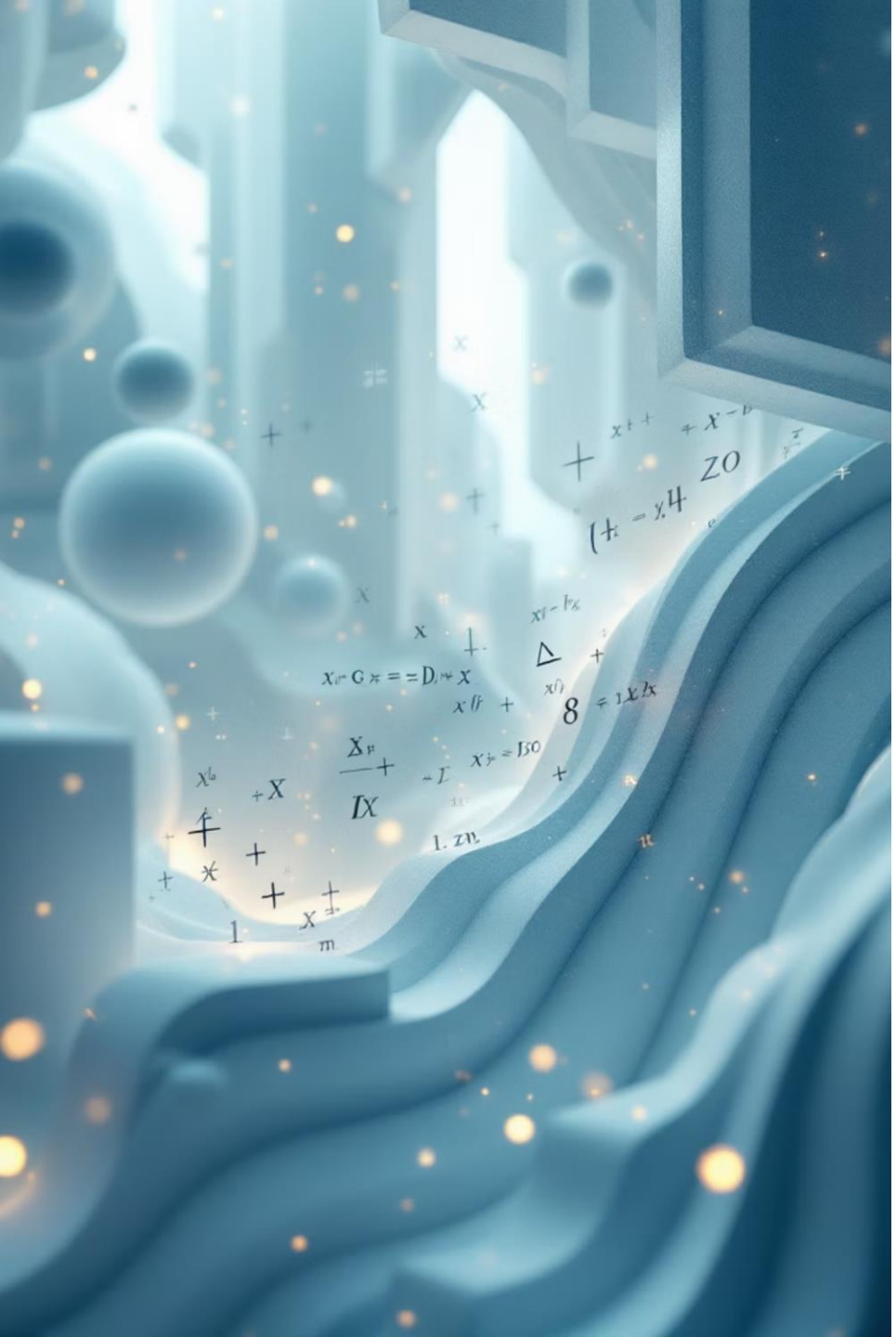


Спецификация программы и доказательство правильности

Один из начальных этапов разработки программы — задание её спецификации. Это описание на логико-математическом языке условий работы и ожидаемого результата программы.

Доказательство правильности программы — её соответствия заданной спецификации — аналогично математическому доказательству, но с дополнительными трудностями, связанными с выполнением программы во времени.





Формальные системы доказательства и ограничения

Потребность в строгости доказательств привела к созданию формальных систем для доказательства правильности программ, источником которых являются дедуктивные системы математической логики.

Однако, существует ограничение на их доказательную силу: согласно теореме Гёделя о неполноте, для любой системы найдутся правильные программы, правильность которых невозможно доказать.



Теорема Гёделя о неполноте

Одно из величайших достижений математической мысли XX века. Если имеется достаточно богатая и непротиворечивая система аксиом, описывающая свойства натурального ряда, и процесс вывода следствий формализован, то всегда найдется истинное утверждение о натуральном ряде, которое не может быть получено из этих аксиом.





Фундаментальное значение теоремы о неполноте

Практически любая математическая теория, включая теоретическое программирование, содержит арифметику как часть. Поэтому теорема о неполноте имеет фундаментальное гносеологическое значение: она показывает неисчерпаемость творческого мышления как средства познания.



Математическая логика и теория вычислительной сложности: влияние на практику

Рассмотрим, как теоретические открытия формируют наше понимание алгоритмов и данных, и обсудим их неочевидное, но глубокое влияние на решение реальных задач.

Начало пути

Эволюция понятия сложности вычисления

Понятие сложности вычисления, изначально возникшее в математической логике, развивалось параллельно и независимо от непосредственных потребностей вычислительной практики. Однако именно эти теоретические изыскания оказали широкое и непрямое влияние на программирование. В частности, это касается так называемых переборных задач, где поиск эффективного алгоритма долгое время оставался центральной проблемой.

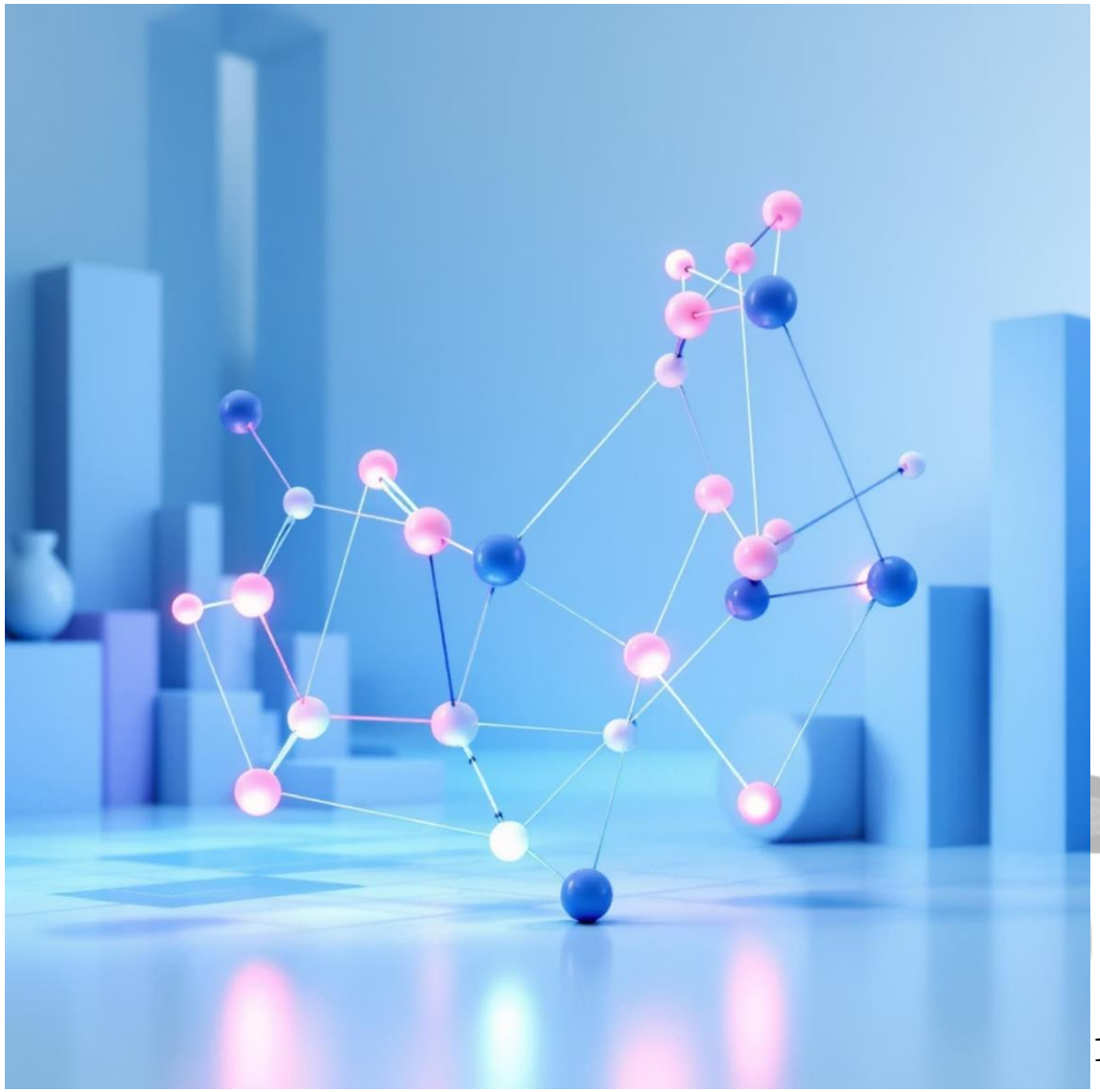


Быстрые алгоритмы

Полиномиальное время и проблема перебора

В теории сложности вычислений "быстрым" считается алгоритм, время работы которого выражается полиномом от объема входных данных. Многие задачи, требующие поиска объекта с заданным соотношением или сигнала о его отсутствии, имеют тривиальный, но медленный алгоритм — полный перебор.

Например, задача о гамильтоновом пути (поиск пути, проходящего через каждую вершину графа ровно по одному разу) является классическим примером такой задачи.



Примеры переборных задач

От графов до оптимизации

1

Задачи на графах

Раскраска графа, поиск максимального паросочетания, задача коммивояжера и другие.

2

Целочисленное линейное программирование

Нахождение оптимальных решений в задачах с линейными ограничениями и целочисленными переменными.

3

Оптимизационные задачи

Широкий класс задач, требующих нахождения наилучшего решения из множества возможных (например, упаковка рюкзака).

На протяжении десятилетий ученые тщетно пытались найти полиномиальные алгоритмы для этих задач. Прорыв произошел не в поиске таких алгоритмов, а в понимании их природы.



Теоретический прорыв

Две ключевые идеи

Эквивалентность переборных задач

Все упомянутые задачи оказались эквивалентны: быстрый алгоритм для одной автоматически дает быстрые алгоритмы для всех остальных. Это понятие стало известно как NP-полнота.

Гипотеза P ≠ NP

Существует нарастающее признание гипотезы о том, что для ни одной из этих задач не существует полиномиального алгоритма. Это одна из нерешенных проблем тысячелетия в математике.

Это сочетание теоремы об эквивалентности и гипотезы стало значительным прогрессом, позволяющим избежать бесплодных поисков.

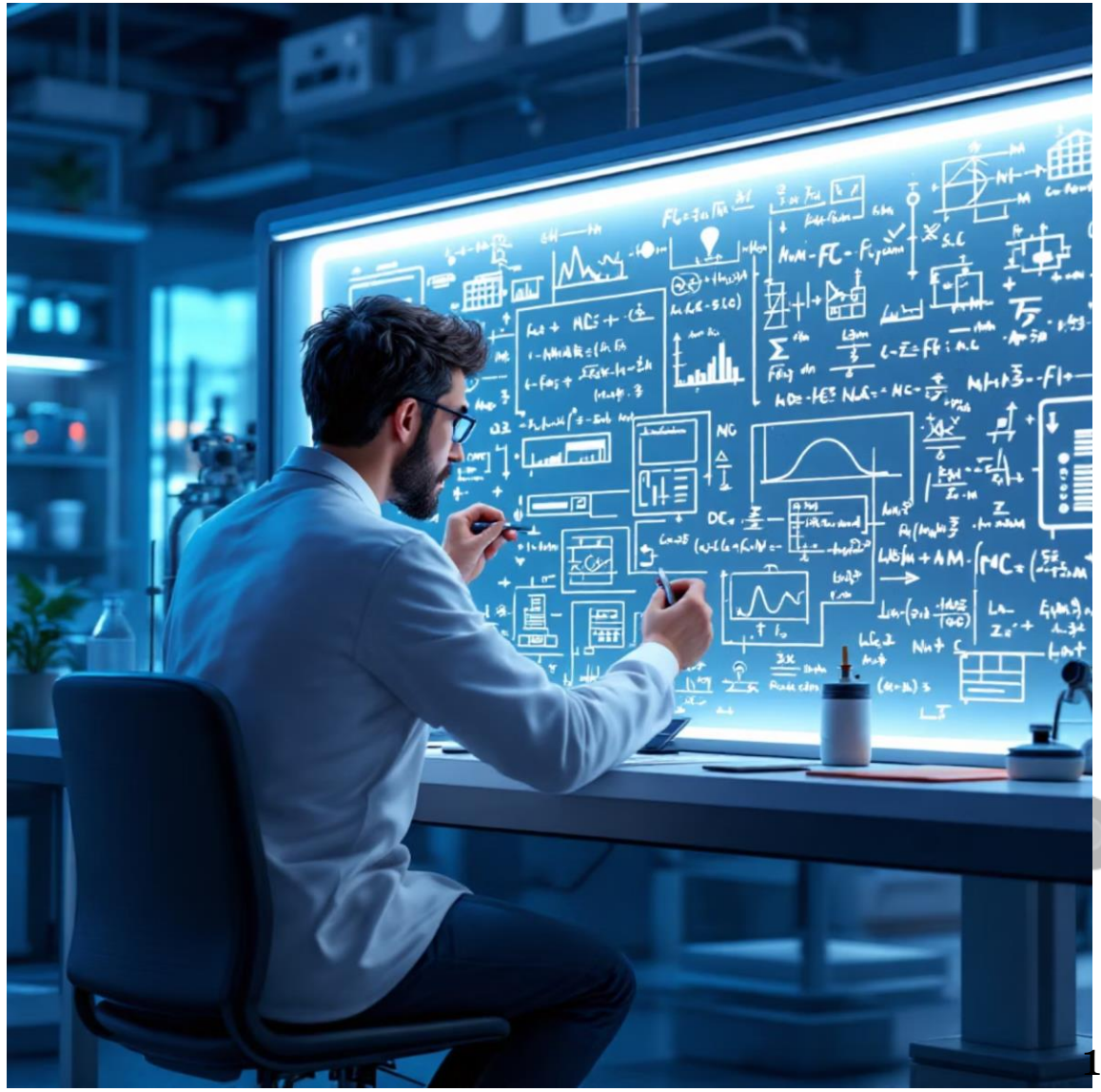


Прикладное значение теории

Новый подход к решению сложных задач

Раньше каждая новая переборная задача приводила к попыткам "кустарным" методом найти для неё быстрый алгоритм. Теперь же, сталкиваясь с такой задачей, ученый может быстро доказать её переборность.

Это позволяет модифицировать формулировку задачи, выделять подклассы, представляющие практический интерес, и уже в них искать эффективные алгоритмы.



Другие результаты

Нижние оценки сложности и сравнение алгоритмов

1 **Доказательство бесплодности**

Теория сложности вычислений дает нижние оценки сложности, показывая, что некоторые алгоритмы фундаментально невозможно улучшить. Это помогает избежать траты ресурсов на бесперспективные направления.

2 **Оценка и сравнение алгоритмов**

Результатом развития теории стало также возможность эффективно оценивать и сравнивать различные конкретные алгоритмы, даже до их полной реализации.

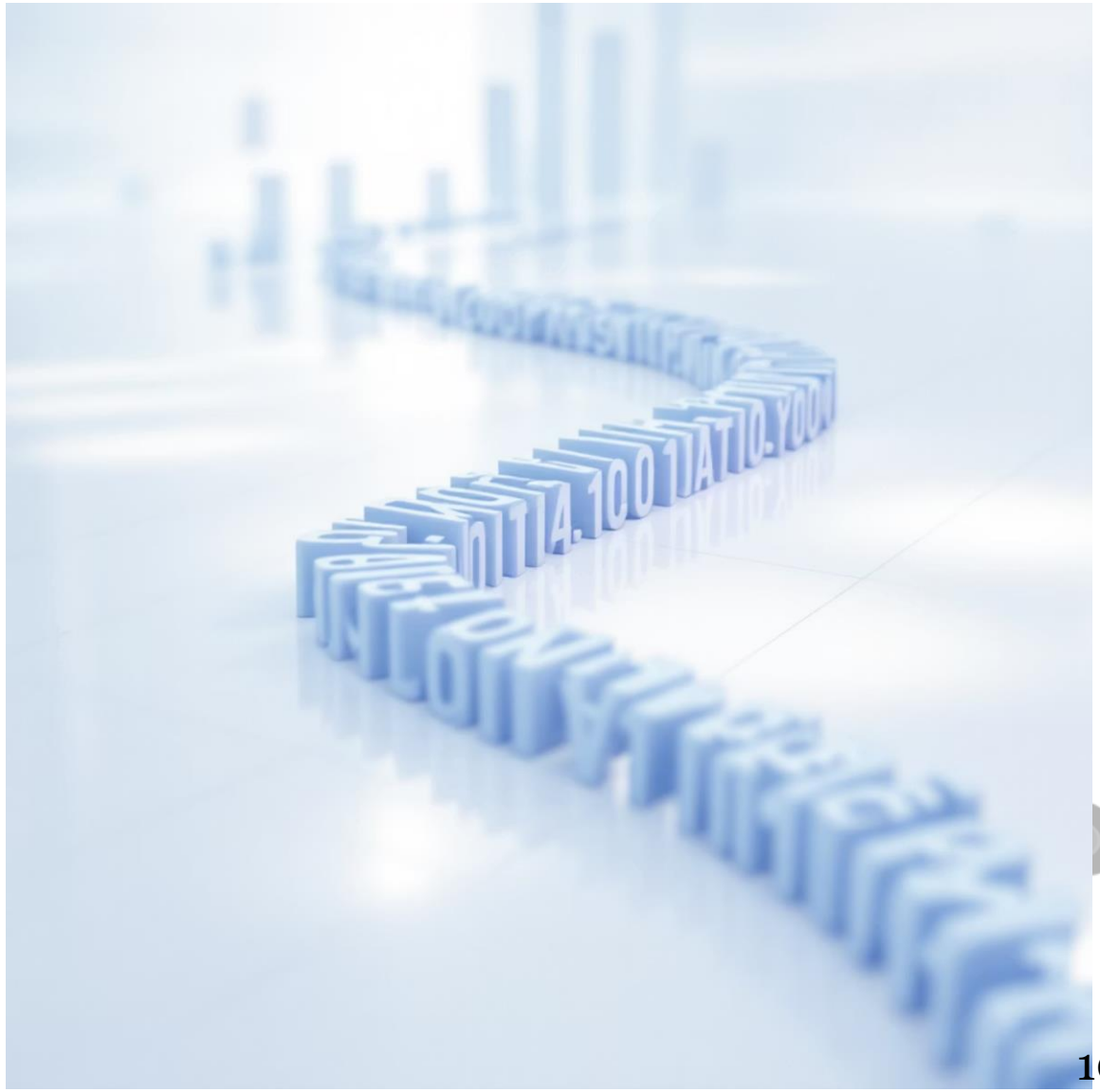


Новая парадигма

Сложность объекта: От описания к энтропии

Понятие сложности объекта (или колмогоровской сложности) — это объем кратчайшего описания объекта при данном способе описания.

А.Н. Колмогоров доказал существование оптимальных (универсальных) способов описания, которые позволяют получать описания не хуже любого другого способа.



Значение отрицательных результатов в программировании

> **Понимание, что в принципе невозможно сделать что-то, имеет для программирования не меньшее значение, чем для техники невозможность нарушить второе начало термодинамики или для физики невозможность превысить скорость света.**

Математическая логика предоставляет критически важные "отрицательные результаты", которые определяют границы вычислимости и возможности программирования.



Ключевые отрицательные результаты

Теорема Гёделя о неполноте

Ограничивает возможность автоматической верификации программ и доказательства их правильности.

Переборные задачи (P vs NP)

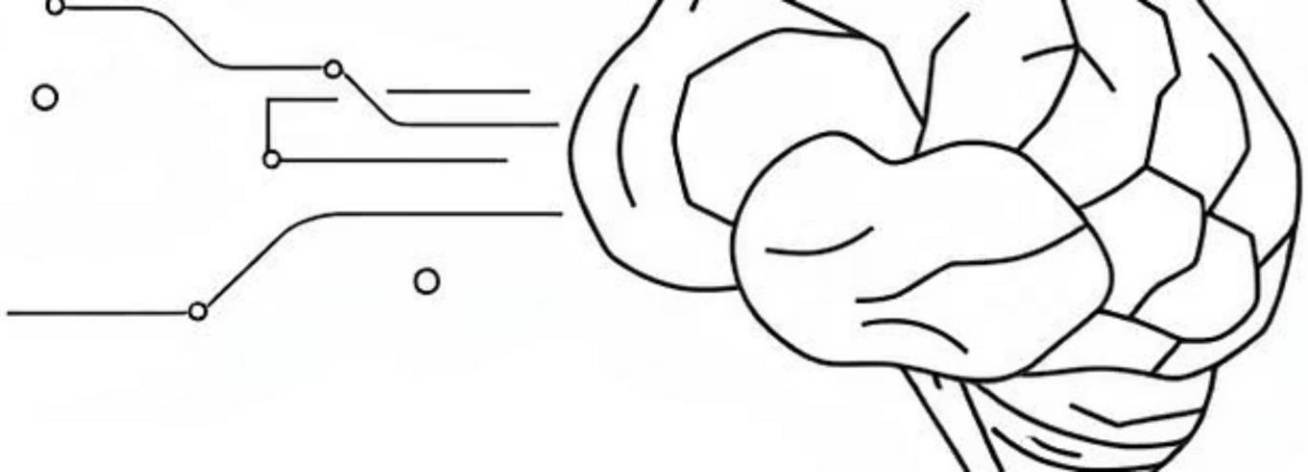
(Условное) доказательство отсутствия быстрых алгоритмов для целого класса задач, что направляет усилия на поиск аппроксимаций или решение специальных подклассов.

Неразрешимость нетривиальных свойств

Например, невозможность создания универсального транслятора, который мог бы определить зацикливается ли программа на любом вводе (проблема остановки).

Эти результаты, хотя и кажутся "отрицательными", на самом деле являются фундаментом для понимания ограничений и возможностей вычислительной практики, направляя дальнейшие исследования и разработки.





Формальные языки: Мост между теорией и практикой

Формальные языки, разработанные в математической логике, оказали значительное влияние на языки программирования и описание работы ЭВМ.

Языковое влияние

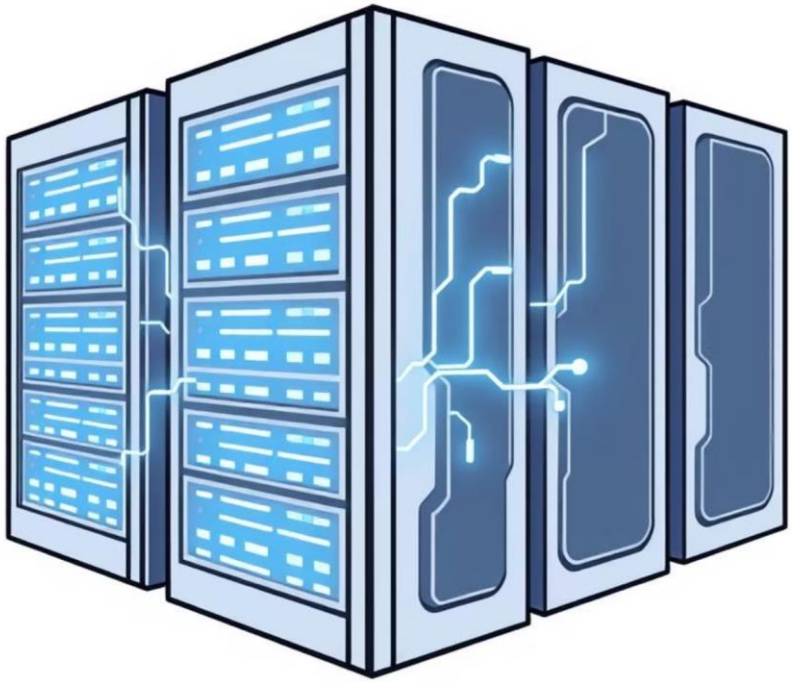
Конкретные логические и алгоритмические языки вошли в программирование.

Концептуальное влияние

Более фундаментально влияние общих языковых концепций и идеи формальной семантики.

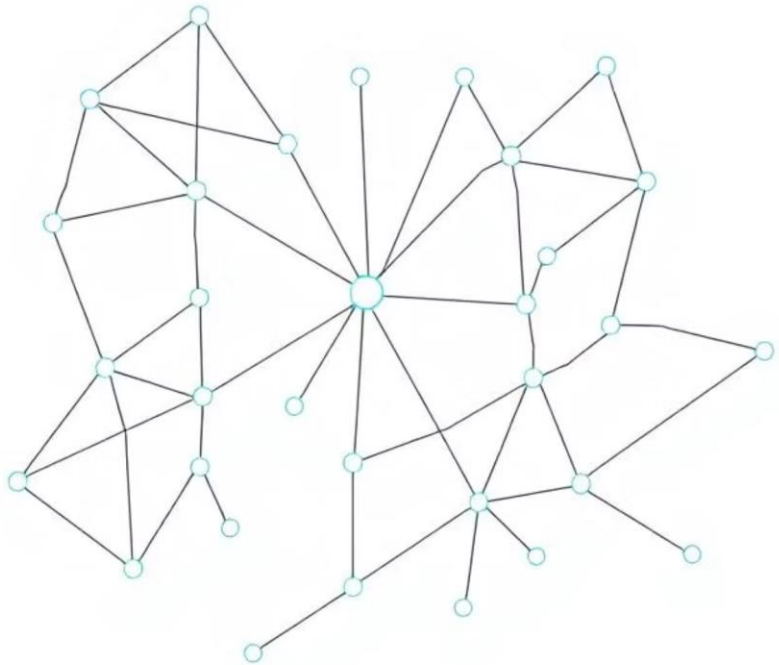
Язык исчисления предикатов: Основа баз данных

Язык узкого исчисления предикатов стал краеугольным камнем в построении современных баз данных, известных как «реляционная модель данных».



- В 70-е годы программисты нуждались в универсальном языке для формулирования утверждений о хранимой информации.
- Исчисление предикатов легло в основу реляционной модели данных.
- Применение не ограничивается базами данных, используется также в системах искусственного интеллекта.





Применение исчисления предикатов в ИИ

Исследования по «естественному выводу» (Г. Генцен, 30-е годы) стали основой для систем автоматического поиска вывода в задачах искусственного интеллекта.

Искусственный интеллект

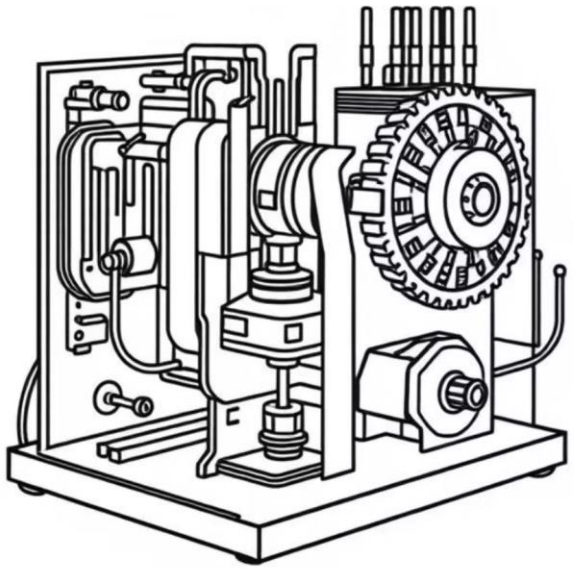
Автоматический поиск вывода в ИИ основан на исследованиях естественного вывода.

Модальная логика

Перспективное направление для ИИ, оперирующее оценками «возможно», «необходимо», «правдоподобно».

Вычислительные модели: от бумаги к реальности

Первые универсальные вычислительные устройства, такие как машина Поста и машина Тьюринга, появились в 1936 году как абстрактные схемы на бумаге.



- Идеализированные абстрактные схемы, не предназначенные для конкретной реализации.
- Умозрительные конструкции и вычислительные реалии тесно взаимодействуют.
- Служат мощным инструментом для исследования принципиальных возможностей реальных вычислительных устройств.



Эволюция абстрактных вычислительных моделей

История абстрактных вычислительных моделей, таких как автоматы фон Неймана – Чёрча и клеточные автоматы, демонстрирует их переход от философских концепций к практической значимости.

1

Начало 50-х

Дж. фон Нейман рассматривает абстрактные вычислительные устройства (философский характер).

2

Развитие теории

Теория развивается, меняются названия: клеточные автоматы, итеративные сети.

3

Современность

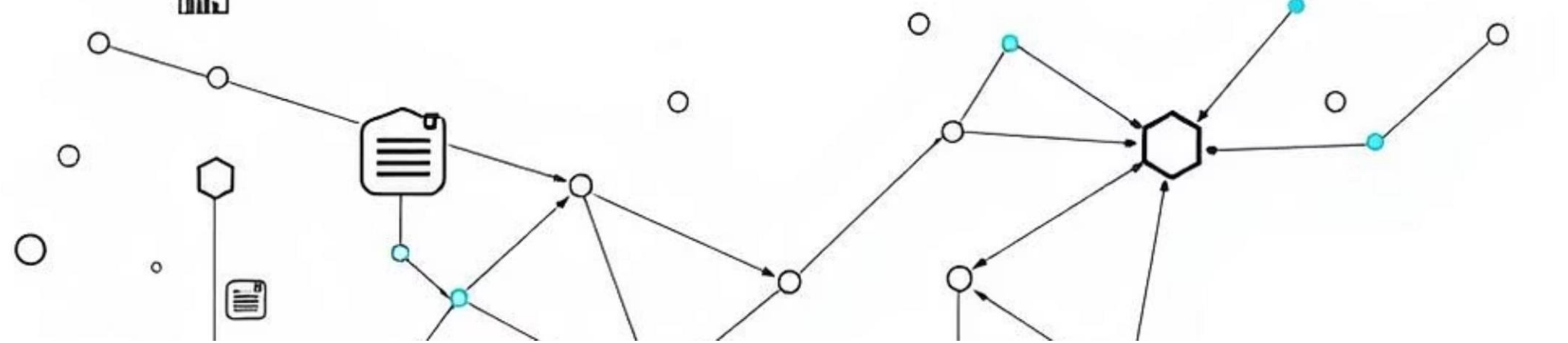
Термин «систолические структуры» наиболее часто употребляется.

4

Практическая значимость

Прогресс в элементной базе вычислительной техники привел к практической значимости.





Алгоритмы и нечисловая информация

Математическая логика поставляет конкретные алгоритмы для работы с формальными языками и конструктивными объектами, что особенно важно для обработки нечисловой информации.

Эффективность

Накоплен значительный запас алгоритмов для работы с символьными цепочками и графами.

Нечисловая информация

Расширение использования вычислительной техники связано с обработкой нечисловой информации.

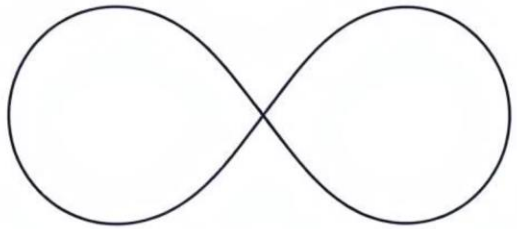
Лингвистический анализ

Алгоритмы синтаксического анализа используют методы математической логики.

Теоремы о неподвижной точке: фундамент рекурсии

Теоремы о неподвижной точке, особенно теорема о рекурсии, играют важную роль в теории алгоритмов и программировании, являясь основой для формальной семантики рекурсивных программ.

- Конкретные теоремы математической логики имеют важное практическое значение.
- Теорема о неподвижной точке вычислимого оператора утверждает существование наименьшей неподвижной точки.



Гуманизация взаимодействия и формализация семантики

Развитие программирования движется к более «человечному» синтаксису, в то время как семантика стремится к большей формализации для создания сложных и надежных программ.

Синтаксис

От жестких правил к неформальному и «человечному».

Семантика

От интуитивных представлений к строгой формализации.

Вызов

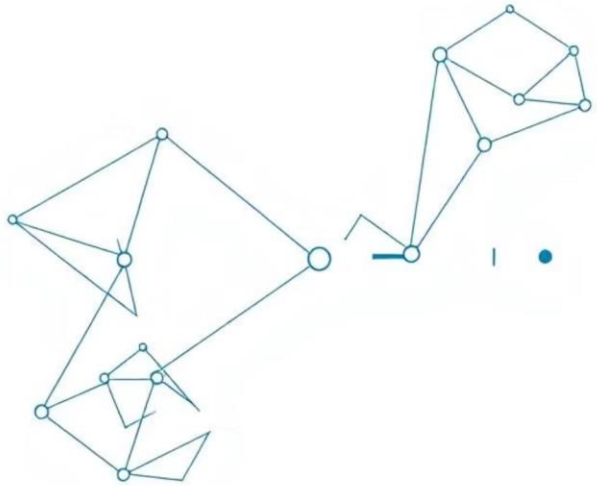
Создание простой, последовательной, строгой и непротиворечивой семантики.



Рекурсивные определения и их семантика

Рекурсивные определения, часто встречающиеся в программировании, позволяют описывать функции через самих себя, что соответствует многоступенчатому разбиению задач.

- Определение функции для объектов со сложной структурой.
- Разбиение объектов на подобъекты для применения функции.
- Автоматическое формирование алгоритма вычисления.
- Неалгоритмический характер семантики рекурсивных определений.





МЛИТА

МАТЕМАТИЧЕСКАЯ ЛОГИКА И ТЕОРИЯ АЛГОРИТМОВ